QUANT INTERVIEW MASTER GUIDE

Dynamic Programming

Seven Problem Families and a Framework for Unfamiliar Problems

Mizuki Seika

From first principles to independently deriving Bellman equations and executable Python code

Recognition clues · Canonical problems · Top-Down / Bottom-Up · Complexity optimization

Audience	From absolute beginners to live coding involving stochastic games and optimization
Learning path	Fundamentals and two implementations → identifying seven families → working through canonical problems → unfamiliar interview cases
Scope	Complete problem statements, recurrences, base cases, code, and complexity analysis, with a clear distinction between deterministic and stochastic DP

© 2026 Mizuki Seika. All original content is copyrighted by the author.

This work is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0).

You are free to share and adapt this material, including for commercial purposes, provided that appropriate credit is given, a link to the license is included, and any modifications are indicated.

Author: Mizuki Seika

Contents

Learning Path and Objectives 1

1 Core Principles: Define the State Before Writing Loops 2

1.1 How Do DP, Backtracking, and Memoization Relate? . . . 2
1.2 Starting from Zero: Why Do We Need DP? . . . 2
1.2.1 From Brute-Force Recursion to Memoization . . . 2
1.2.2 From Memoization to Direct Table Filling . . . 3
1.3 Top-Down and Bottom-Up: Two Essential DP Implementations . . . 4
1.3.1 Top-Down: Ask the Large Question First, Then Recurse on Smaller Ones . . . 4
1.3.2 Bottom-Up: Start with Known Subproblems and Follow Dependencies . . . 4
1.3.3 When Is Adding @cache to DFS Not Enough? . . . 5
1.3.4 Distinguishing Evaluation Direction from Backward Induction . . . 5
1.4 Deriving a DP State from a Problem Statement . . . 5
1.5 Three Kinds of Base Cases and Tiny Hand-Checked Tests . . . 6
1.6 Bellman Equations: From Actions to Values . . . 6
1.7 Four Common Ways to Combine Subproblems . . . 6
1.8 How to Prove a State Transition Correct . . . 7
1.9 A Unified Complexity Calculation . . . 7

2 Family One: Optimal Stopping and Expected-Value Games 8

2.1 Recognition Clues, Structure, and Core Technique . . . 8
2.2 Recognition in Detail: Optimal Stopping or Just a Probability Problem? . . . 8
2.2.1 Four Distinct Concepts: Outcome, Action, State, Value . . . 9
2.2.2 Always Specify the Timing of Decisions in a Recurrence . . . 9
2.2.3 A Reusable Interview Answer for Stopping Problems . . . 9
2.3 Canonical Problem 1: Up to Three Rolls of a Fair Die . . . 9
2.4 Canonical Problem 2: Red/Black Sampling without Replacement and Optimal Stopping . . . 11
2.5 Canonical Problem 3: An Infinite-Horizon Die Game with Ruin on Six . . . 13
2.6 Canonical Problem 4: Replicating a Series Payoff with Bets (Binomial Hedging) . . . 14
2.7 Algorithm Selection and Checks for This Family . . . 15

3 Family Two: Sequential Decisions and State Machines 17

3.1 Structure and Recognition: How Historical Rules Change Your Current Status . . . 17
3.2 Recognition in Detail: When Must We Remember Yesterday's Status? . . 17
3.2.1 Drawing a State Machine by Hand on a Tiny Example . . . 18
3.2.2 Do Not Mix Up Two Definitions of Subproblems . . . 18
3.2.3 Manual Example: Why a Cooldown Splits the State . . . 18
3.3 Canonical Problem 1: House Robber — No Adjacent Selections . . . 18
3.4 Canonical Problem 2: Kadane's Algorithm — Maximum Subarray Sum . 20
3.5 Canonical Problem 3: Maximum Product Subarray — Track the Minimum Too . . . 21
3.6 Canonical Problem 4: At Most K Completed Stock Trades . . . 22
3.7 Canonical Problem 5: Stock Trading with a Cooldown . . . 24
3.8 Advanced Variations: Additional States and Rolling Optimization . . . 25

4 Family Three: Resource Allocation and Knapsack 26
4.1 Recognition: Take or Skip, and How Does Capacity Change? . . . 26
4.2 Recognition in Detail: Knapsack Is Constrained Selection, Not Literally Packing a Bag . . . 26
4.2.1 Start with a Two-Dimensional Table to Get the Intuition Right . . . 27
4.2.2 Why Does One-Dimensional Compression Change the Capacity Order? . . . 27
4.2.3 Initialization Comes from Problem Semantics, Not Memorized Code 27
4.3 Canonical Problem 1: Standard 0/1 Knapsack . . . 28
4.4 Canonical Problem 2: Partition Equal Subset Sum (LeetCode 416) . . . 30
4.5 Canonical Problem 3: Coin Change (LeetCode 322) . . . 32
4.6 Canonical Problem 4: Combination Counts (LC 518) versus Ordered Sequences (LC 377) . . . 33
4.7 Canonical Problem 5: Target Sum (LeetCode 494) . . . 35
4.8 Advanced Models: Bounded, Group, and Multidimensional Knapsack . . 36

5 Family Four: Two-Sequence Alignment and Grid Paths 39
5.1 Recognition: How Do Two Indices or Coordinates Advance? . . . 39
5.2 Recognition in Detail: Why Do Two Indices Often Mean Two-Dimensional DP? . . . 39
5.2.1 Two Sequences: How Do the Final Characters Partition the Problem? . . . 40
5.2.2 Grid Paths: Entering versus Leaving a Cell . . . 40
5.2.3 Which Row Does a Rolling One-Dimensional Array Actually Store? 40
5.3 Canonical Problem 1: Unique Paths (LeetCode 62) . . . 40

5.4 Canonical Problem 2: Longest Common Subsequence (LCS, LeetCode 1143) . . . 42
5.5 Canonical Problem 3: Edit Distance (LeetCode 72) . . . 44
5.6 Canonical Problem 4: Regular Expression Matching (LeetCode 10) . . . 45
5.7 Canonical Problem 5: Catalan Numbers and Dyck Paths (Diagonal Constraint) . . . 46
5.8 Supplementary Canonical Problem: Word Break (LeetCode 139) . . . 47

6 Family Five: Interval Merging and Zero-Sum Games 49
6.1 Recognition Clues and Common Structure . . . 49
6.2 Recognition in Detail: Is the Remainder One Interval or an Arbitrary Set? 49
6.2.1 How Is Interval DP Different from Ordinary Prefix DP? . . . 50
6.2.2 Net Score Difference in Zero-Sum Games: Why Is It Simpler Than Tracking Player Totals? . . . 50
6.2.3 Why Does Interval DP Usually Iterate over Length First? . . . 50
6.2.4 Choosing Boundaries Carefully: Closed versus Open Intervals . . . 50
6.3 Canonical Problem 1: Taking Coins from the Ends (Predict the Winner, LeetCode 486) . . . 50
6.4 Canonical Problem 2: Burst Balloons (LeetCode 312) . . . 52
6.5 Canonical Problem 3: Longest Palindromic Subsequence (LeetCode 516) 54
6.6 Canonical Problem 4: Matrix Chain Multiplication . . . 55

7 Family Six: State Compression with Bitmask DP 57
7.1 Recognition: The Set Is Small, but Exactly Which Elements Were Used Matters . . . 57
7.2 Recognition in Detail: Why Must the Set Itself Be a State? . . . 57
7.2.1 What Is a Bitmask? A Four-Element Example . . . 58
7.2.2 When Is mask Insufficient, and Why Add last? . . . 58
7.2.3 Estimate State Counts Rather Than Memorizing Formulas . . . 58
7.3 Canonical Problem 1: Travelling Salesperson Problem (TSP) . . . 58
7.4 Canonical Problem 2: Minimum-Cost Assignment . . . 60
7.5 Canonical Problem 3: Smallest Sufficient Team . . . 62
7.6 Algorithmic Limits of This Family . . . 63

8 Family Seven: Digit DP, Finite Automata, and Markov Counting 64
8.1 Why Do These Problems Belong Together? . . . 64
8.2 Recognition in Detail: Which Small Piece of History Does a Long Sequence Need? . . . 64
8.2.1 Compressing Complete Histories into a Finite State . . . 65
8.2.2 The Meaning of tight in Digit DP . . . 65
8.2.3 Leading Zeros and the Boundary for Zero or Negative Numbers . 65

8.3 Canonical Problem 1: Never Toss Three Consecutive Heads 65
8.4 Canonical Problem 2: Markov Transitions and Matrix Exponentiation . . 67
8.5 Canonical Problem 3: Digit DP — Binary Integers without Adjacent Ones in $0 \le x \le N$ 68
8.6 When Is started Required for Leading Zeros? 70
8.7 Canonical Problem 4: Knight Dialer (LeetCode 935) 70
8.8 Advanced Pattern Matching: What Does an Automaton State Represent? 71

9 Four Steps for Unfamiliar Problems: From Words to Recurrences to Optimization 72
9.1 Step One: Use Input Constraints to Estimate an Acceptable Complexity 72
9.2 Step Two: Use the Amnesia Test to Find a Minimal Sufficient State . . . 72
9.3 Step Three: Write the Natural Recursive Actions, Then Cache Repeated States 73
9.4 Step Four: Convert to Bottom-Up and Reduce Space 73
9.5 Five Trap-Like Unfamiliar Questions: Recognizing States on the Spot . . 74
9.6 From Mathematical Recurrence to Submission-Ready Code: How to Explain It 74
9.7 An Eight-Step Checklist for Every Unfamiliar Interview Problem 75
9.8 Complete Unfamiliar Quant Case: Restricted High-Frequency Trading Decisions 75
9.8.1 Restating the Problem and Mathematical Model 75
9.8.2 Step 1: Use the Input Size to Target an $O(n)$ State Machine 76
9.8.3 Step 2: The Amnesia Test 76
9.8.4 Step 3: Turn Brute-Force Actions into Memoized Recursion 76
9.8.5 Step 4: Roll Five Forward States into $O(1)$ Space 77
9.8.6 Interview Validation: Brute-Force Small Samples 77
9.9 Troubleshooting: Six Ways to Get Stuck in an Interview 78

A Interview Quick Reference: Seven DP Families 79
A.1 Complexity Quick Reference 79

B Pre-Interview Self-Test: From Recognition Clues to Code 81
B.1 Ten Quick Questions You Must Be Able to Answer without Coding 81
B.2 Suggested Answers to the Ten Questions 81

C General Python Templates and Self-Checks 83
C.1 Top-Down Optimal-Value Template 83
C.2 Bottom-Up State-Machine Template 83
C.3 Pre-Submission Checklist 84

Learning Path and Objectives

This guide provides systematic preparation for live algorithm interviews for Quantitative Researchers and Traders. Dynamic programming (DP) is not one fixed template: it is a method for **compressing the information needed for future decisions into a state and reusing answers to overlapping subproblems**. By the end, you should be able to define a minimal sufficient state, justify transitions, specify base cases, implement and optimize a solution, and explain its complexity and limitations for unfamiliar problems.

Problem family	Typical recognition clues	State representation
1 Optimal stopping and expectation	Dice, sampling without replacement, continue versus stop, expected payout	$V(t, x)$ or $V(r, b)$
2 Sequential decisions and state machines	Positions, cooldowns, streak lengths, transaction fees	$dp[t]$[position][k]
3 Resource allocation and knapsack	Usage limits, budgets, target sums, counting ways	$dp[i][c]$ or $dp[c]$
4 Two sequences and grids	Editing, alignment, two-dimensional position, path constraints	$dp[i][j]$
5 Intervals and zero-sum games	Taking ends, bursting balloons, merging intervals	$dp[l][r]$
6 Bitmask state compression	Small n, visited sets, assignments/TSP	$dp[mask][last]$
7 Digit and Markov counting	Runs of symbols, restricted integers, long-horizon transitions	$dp[t][s]$ or $dp[pos][tight][state]$

Each chapter follows the same structure: recognition clues → state definition → recurrence derivation → full canonical problem and example → straightforward code → optimizations and complexity → common mistakes. Chapter Eight synthesizes these methods into a four-step approach for unfamiliar problems.

1 Core Principles: Define the State Before Writing Loops

1.1 How Do DP, Backtracking, and Memoization Relate?

Backtracking explores branches of valid decisions; DP solves **repeated subproblems with the same state** only once. Adding `@cache` to recursion can turn it into top-down DP, provided that the cache key contains all information affecting future feasibility and reward. If the task requires listing every path, or two histories have different constraints despite reaching the same location, caching only by location can be incorrect.

Key Insight

The amnesia test: Imagine standing at step t with no memory of how you got there. Which variables would you minimally need written on a note to determine whether the next action is legal and what reward it yields? These are your state variables. Any historical constraint affecting the future must be included in the state: completed trades, consecutive days, visited sets, previous actions, and so on.

1.2 Starting from Zero: Why Do We Need DP?

When a problem asks for the best choice among many possibilities, the most direct method is to try all choices. For instance, suppose a staircase has n steps and each move climbs either one or two steps. How many distinct ways can we reach the top? Let $F(n)$ count the ways to climb n steps. The final move must be either one or two steps, so

$$F(n) = F(n-1) + F(n-2), \qquad F(0) = F(1) = 1.$$

This recurrence is not a formula to memorize. It follows from **partitioning all solutions by the final move into two exhaustive and disjoint groups**. Directly computing $F(5)$ recursively requires both $F(4)$ and $F(3)$; computing $F(4)$ evaluates $F(3)$ again. As n grows, identical subproblems are solved repeatedly.

Key Insight

The first motivation for DP is **overlapping subproblems**: different decision paths lead to the same smaller problem. The second requirement is **reusability**: once the state is known, future valid choices and answers do not require the full past history. These conditions are more fundamental than merely seeing "maximize" or "minimize" in a question.

1.2.1 From Brute-Force Recursion to Memoization

Without caching, the recursion tree can become very large:

```python
def stairs_slow(n):
    if n <= 1:
        return 1
    return stairs_slow(n - 1) + stairs_slow(n - 2)
```

When the **same arguments always describe the same subproblem and yield the same result**, we can cache previously computed answers:

```python
from functools import cache

@cache
def stairs_topdown(n):
    if n <= 1:
        return 1
    return stairs_topdown(n - 1) + stairs_topdown(n - 2)

assert stairs_topdown(5) == 8
```

For $n = 5$, only the six states $n = 0, 1, 2, 3, 4, 5$ need to be evaluated; each is computed at most once, reducing time to $O(n)$. A recursive call stack remains, so space is still $O(n)$.

1.2.2 From Memoization to Direct Table Filling

Since $F(n)$ depends only on $F(n-1)$ and $F(n-2)$, we can compute small cases first and progressively build larger ones. This is called bottom-up DP:

```python
def stairs_bottomup(n):
    if n <= 1:
        return 1
    dp = [0] * (n + 1)
    dp[0] = dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp[n]
```

This code uses **exactly the same mathematical state and recurrence** as the recursive version. Only the evaluation order differs. Its time and space are still $O(n)$. Because only two previous values are needed, we can replace the entire array with two variables:

```python
def stairs_rolling(n):
    a, b = 1, 1
    for _ in range(2, n + 1):
        a, b = b, a + b
    return b
```

Space is now $O(1)$. These three versions give a common progression for the DP problems in this book: **discover the structure recursively → cache repeated work → fill a table without recursion → use rolling states to save space**.

1.3 Top-Down and Bottom-Up: Two Essential DP Implementations

1.3.1 Top-Down: Ask the Large Question First, Then Recurse on Smaller Ones

Top-down starts at the state whose answer we want. For $F(5)$, the code asks for $F(5)$, which asks for $F(4)$ and $F(3)$; recursion stops upon reaching a base case. Computed values are stored in a cache and returned immediately on later visits.

When implementing top-down DP, check the following in order:

1. **Function arguments:** Does `dfs(i, capacity, last)` contain every piece of information needed for the future? Histories that "look identical" under these arguments must truly have identical remaining subproblems.
2. **Base case:** What should be returned at the end? Should an infeasible state return $-\infty$, $+\infty$, False, or 0?
3. **Actions:** What legal choices are available? How does each choice change the state and immediate reward?
4. **Cache:** Is the state hashable? Is the number of states manageable? Can the recursion contain cycles?

For House Robber, `dfs(i)` is the maximum payoff obtainable from index i onward. Its two branches are "skip i" and "take i, then skip $i+1$." For stochastic games, first probability-weight the payoffs across random outcomes for a given action, then optimize over the actions the player can choose.

1.3.2 Bottom-Up: Start with Known Subproblems and Follow Dependencies

Bottom-up is not a different mathematical algorithm; it evaluates the same state recurrence in another order. We build a table starting from **boundary states** that require no recursive calculation, and every step reads only values already computed. If $dp[i]$ depends on $dp[i-1]$, iterate i upward; if interval $dp[l][r]$ depends on shorter intervals, iterate by increasing interval length.

The two approaches compare as follows:

Dimension	Top-Down + Cache	Bottom-Up Table Filling
Implementation	Recursive `dfs(state)` calls	Ordered `dp[state]` loops
States visited	Only states reachable from the initial query	May enumerate states never needed
Termination	Base cases stop recursion	Initialize boundary values, then fill in dependency order
Time	Number of states × work per state	Number of states × work per state
Space	Cache plus recursion stack	Array/dictionary; rolling states possible

Dimension	Top-Down + Cache	Bottom-Up Table Filling
Best suited for	Complex rules, sparse states, natural action descriptions	Large n, deep recursion risks, constant-factor optimization

Important: Their asymptotic time complexities are often the same, but *not always*. Top-down can compute fewer states when few are reachable; a cache key containing unnecessary history can also produce exponentially many states.

1.3.3 When Is Adding @cache to DFS Not Enough?

Suppose we seek a maze path that **visits each cell at most once**. Two histories reaching the same cell (i, j) might have visited different cells, leaving different legal future moves. Caching only `dfs(i,j)` incorrectly merges distinct subproblems. An exact state also needs `visited`, potentially creating 2^{mn} possible visited sets.

Another example is when we must **enumerate every concrete path**, rather than compute the optimal value or number of paths. Caching a single optimal value cannot by itself emit different path prefixes. For infinite-horizon stochastic problems with cyclic state transitions, recursion plus caching need not terminate; we may have to analyze convergence, stopping, or solve a fixed-point equation iteratively.

1.3.4 Distinguishing Evaluation Direction from Backward Induction

Top-down and bottom-up describe **how code evaluates subproblems**. Backward induction describes **the mathematical idea of deriving present optimal values from known future terminal values**. In a dice game we reason backward from the last opportunity; we can nevertheless implement that recurrence as top-down recursion `dfs(remaining)` or a bottom-up loop `for remaining in range(1,N+1)`. The notions do not conflict.

1.4 Deriving a DP State from a Problem Statement

A beginner's most common obstacle is not knowing how to write a `for` loop; it is not having answered **"Which smaller problem does this optimal value describe?"** Start by rewriting the task in one sentence: *Given only this essential information, what is the best outcome I can obtain from this point onward?*

Distinguish three definitions that are easily confused:

- **Prefix optimal:** $dp[i]$ is the best value obtainable from the first i elements, as in House Robber; the answer is usually $dp[n]$.
- **Must end at this position:** $dp[i]$ is the best value *required to end at element* i, as in Maximum Subarray or LIS; the answer is usually $\max_i dp[i]$, not simply the last entry.
- **Exact target:** $dp[c]$ represents the optimal value or feasibility of *reaching ca-*

pacity/amount c exactly, as in Coin Change; unreachable states need an explicit sentinel.

Arrays all called `dp[i]` can answer completely different questions. Once the state is defined, partition the problem by its **last step**, **first step**, or **current choice**. In knapsack, the last item is either taken or skipped; in edit distance, the last characters are aligned, inserted, removed, or replaced.

1.5 Three Kinds of Base Cases and Tiny Hand-Checked Tests

Optimal value: If selecting nothing is allowed, the empty state's payoff is often 0; if at least one element must be selected, initializing the empty state to 0 without thought can give a wrong answer for all-negative arrays. **Feasibility:** The empty target is usually `dp[0]=True`, with other states initially False. **Counting:** The empty combination usually has *one* construction, so `dp[0]=1`; if ordering should not matter, the iteration must avoid double-counting.

For every problem, check tiny examples first: empty input (if permitted), one element, two elements, an unreachable target, exactly one possible path, all-negative values, and answers requiring zero actions. Verify whether the return value is the **last state** or the **maximum over all states**. This often saves more time than fixing index errors after submission.

1.6 Bellman Equations: From Actions to Values

Deterministic maximization:

$$V(s) = \max_{a \in A(s)} \{r(s,a) + V(T(s,a))\}.$$

Stochastic maximization:

$$V_t(s) = \max_{a \in A_t(s)} \mathbb{E}\left[r_t(s,a,\xi) + V_{t+1}(T(s,a,\xi)) \mid s,a\right].$$

Optimal stopping:

$$\boxed{V(s) = \max\{\text{Stop}(s),\ \mathbb{E}[V(S') \mid s, \text{Continue}]\}.}$$

The comparison must use the **value under the optimal future policy**, not merely the expected outcome of the next single random draw. With risk neutrality one may maximize expected money; with risk aversion use expected utility, potentially adding wealth to the state.

1.7 Four Common Ways to Combine Subproblems

- **Max/Min:** Choose the best action, as in knapsack, stock trading, and edit distance.
- **OR/AND:** Decide whether a valid solution exists, as in Word Break or Subset Sum.

- **Sum:** Count solutions from alternative branches, distinguishing combinations from permutations.
- **Probability-weighted sum:** Compute an expectation or probability distribution of a stochastic process, and only then maximize/minimize over *selectable actions*.

1.8 How to Prove a State Transition Correct

A common proof has three steps: (1) partition all feasible solutions into disjoint cases by the current or final action; (2) show that the remaining part of each case is exactly one of the defined subproblems; and (3) argue that a nonoptimal remainder could be replaced by a better one, contradicting optimality (the principle of optimality). Counting problems additionally require no omissions or duplicate counting. The boundary condition must agree with the state definition: `0`, `False`, $+\infty$, and $-\infty$ are not interchangeable.

1.9 A Unified Complexity Calculation

$$\boxed{\text{Time} \approx \#\text{reachable states} \times \text{work per state.}}$$

Space includes caches or tables, the recursion stack, and any predecessor pointers required to reconstruct the optimal policy. When $n \leq 20$, 2^n-state bitmask DP is worth considering but not guaranteed; when $n \sim 10^5$, a near-linear algorithm is usually needed. **Input bounds are clues about acceptable complexity, not mathematical definitions of states.**

2 Family One: Optimal Stopping and Expected-Value Games

2.1 Recognition Clues, Structure, and Core Technique

Clues: "try once more," "when should I stop," "random outcome each round," "maximize expected winnings," "draw without replacement," and "American option." Common states include attempts remaining, accumulated wealth, and remaining red/black objects; actions may be stop/continue or continuous controls such as bet sizes.

Core technique: First identify the order of information. Rolling a die, observing the outcome, and then deciding whether to stop is different from choosing a bet before observing its random outcome: the order of the maximum and expectation changes. Specify terminal payoffs and work backward from the final state, where information is greatest. Infinite-horizon problems require absorbing states, a stopping threshold, or a Bellman fixed point.

$$V_t(s) = \max\{S_t(s),\ \sum_{s'} P(s' \mid s) V_{t+1}(s')\}.$$

Here V can represent total wealth or *incremental payoff from the present onward*; once you choose a definition, do not double-count gains in the transition.

2.2 Recognition in Detail: Optimal Stopping or Just a Probability Problem?

Seeing "dice," "lottery," or "option trading" does not automatically mean DP is needed. Determine **when actions occur, when information becomes available, and whether there is a right to wait or exit early**. The following formulations are especially revealing:

1. **Stop after observing:** "You may roll up to three times; after seeing a roll, keep its number or give it up and roll again." After every new observation you can make another choice, so future value is not merely the mean of one die roll. Remaining attempts form the state; once the current roll is visible, compare its immediate stopping payoff with the expected value of the next stage.

2. **Cash out accumulated winnings at any time:** "You currently have $10; another roll might add money or wipe everything out." Existing wealth now affects the risk of continuing. The state must include *current wealth*; remaining attempts alone are insufficient.

3. **Sampling without replacement:** "The numbers of red and black balls left in the box change." The probability of the next outcome is no longer fixed at 50%; the *remaining count of each type* is the state. The drawing order is unnecessary unless outcomes change additional rules.

4. **Choose the bet size, not just stop/continue:** "Choose a stake each round to replicate a specified final payoff, no matter how the games turn out." The action is a continuous wager amount. You may need to solve linear equations that hold for every branch, rather than simply compare stopping with continuing.
5. **Discounting, fees, or risk aversion:** "Each round of waiting costs \$1," "discount the payoff at the interest rate," or "maximize expected utility instead of money." Include these effects in the stage reward or redefine value; a threshold from the no-cost version no longer applies automatically.

2.2.1 Four Distinct Concepts: Outcome, Action, State, Value

Consider "You observed a 5. Do you roll again?" The face value 5 is an observed outcome; *continue* is an action. If two opportunities remain, the combination of "two remaining attempts" and "current face 5" determines the decision. Value is the conditional expected amount obtainable under the optimal future policy. Computing only the next roll's mean of 3.5 ignores the valuable option to reroll again afterward.

2.2.2 Always Specify the Timing of Decisions in a Recurrence

If action a must be chosen *before* random outcome ξ is revealed, then

$$V(s) = \max_a \sum_\xi p(\xi \mid s, a)\big[r(s, a, \xi) + V(s')\big].$$

If one can *observe* the random outcome before selecting the action, optimize separately for each observation and then take the expectation over observations:

$$\mathbb{E}_\xi \left[\max_a Q(s, \xi, a)\right].$$

These expressions cannot be interchanged at will. For the same available actions, $\mathbb{E}[\max_a Q] \geq \max_a \mathbb{E}[Q]$: observing before acting has informational value. This explains why some dice games have a fair entry value above the expectation of a single roll.

2.2.3 A Reusable Interview Answer for Stopping Problems

Begin with "I compress the currently relevant information into state s and define $V(s)$ as the optimal expected total payoff from here." Then specify "stopping earns $S(s)$; continuing leads to these next states with these probabilities." Write $V(s) = \max(S(s), C(s))$; establish base cases and work backward along the process's dependencies. If states can cycle or the horizon is infinite, justify termination, the threshold, or the need for value iteration. **Also report the policy**: when to stop or continue in each state, not only the entry price.

2.3 Canonical Problem 1: Up to Three Rolls of a Fair Die

Full problem statement: When is information revealed?

You buy a ticket to a game using a fair six-sided die. You must roll once and may roll at most three times in total. After observing a result, you can either stop and

collect that many dollars or discard the result and roll again. The same choice is available after the second roll; after the third roll you must accept the result. Rolls are independent, and no extra fee is charged for another roll. Determine (1) the risk-neutral fair entry price, (2) which observed faces justify stopping on the first and second rolls, and (3) the extension to at most N rolls.

The crucial distinction. If you have just rolled a 4, taking \$4 is the *stopping payoff*. Discarding that 4 with two more opportunities available has a *continuation value*; it is not simply the mean of the next die roll, because a poor second result may itself be discarded.

Complete backward calculation

With only one roll left, the expected payoff is $(1+2+3+4+5+6)/6 = 3.5$. After the second roll, stop on $4, 5, 6$, but roll again on $1, 2, 3$. Consequently the value *before* the second roll is $(3.5+3.5+3.5+4+5+6)/6 = 4.25$. After the first roll, continue on $1, 2, 3, 4$ and stop on $5, 6$. The initial value is $(4\cdot 4.25+5+6)/6 = 14/3$. Each decision is made *after* observing its own outcome; we have not committed in advance to stopping after a fixed number of rolls.

In the iterative code, `value` denotes the optimal expected payoff before observing the next roll when fewer opportunities remain. The expression `max(x, value)` chooses optimally after observing face x, and the outer mean averages over the six possible outcomes. Check that one roll gives 3.5 and three rolls give $14/3$. With an unfair die, replace the uniform average by probability weights. The operative order is $\mathbb{E}[\max(\cdot)]$, not an unjustified $\max(\mathbb{E}[\cdot])$.

Problem: You may roll a fair six-sided die up to three times. After seeing a face value $x \in \{1, \ldots, 6\}$, either accept x dollars and stop, or forfeit it and roll again. The third roll must be accepted. Find the fair entry value and optimal policy at each stage.

Backward induction: The expected payoff of the last roll is $A_1 = (1+\cdots+6)/6 = 3.5$. After the second roll shows x, take $\max(x, 3.5)$, so the value before that roll is

$$A_2 = \frac{1}{6}[3(3.5) + 4 + 5 + 6] = 4.25.$$

After the first roll shows x, take $\max(x, 4.25)$, giving

$$A_3 = \frac{1}{6}[4(4.25) + 5 + 6] = \boxed{14/3 \approx 4.6667}.$$

Optimal policy: Stop on $5, 6$ after the first roll; stop on $4, 5, 6$ after the second; the third roll is compulsory. With any m opportunities:

$$A_1 = 3.5, \qquad A_m = \frac{1}{6}\sum_{x=1}^{6} \max(x, A_{m-1}); \quad \text{Stop if } x \geq A_{m-1}.$$

```python
def dice_value(rolls: int) -> float:
    if rolls <= 0:
        return 0.0
    value = 3.5
    for _ in range(2, rolls + 1):
        value = sum(max(x, value) for x in range(1, 7)) / 6
```

```
    return value

assert abs(dice_value(3) - 14 / 3) < 1e-12
```

Complexity: $O(6N) = O(N)$ time and $O(1)$ auxiliary space. **Follow-ups:** For face probabilities p_x, replace $1/6$ with p_x; with rerolling fee c, continuation value becomes $A_{m-1} - c$.

2.4 Canonical Problem 2: Red/Black Sampling without Replacement and Optimal Stopping

Full problem statement: Sampling without replacement

A bag contains R red balls and B black balls. Draw one ball uniformly at random, *without replacement*, whenever you decide to continue. A red ball adds \$1 to your cumulative winnings; a black ball subtracts \$1. After each draw you may stop and collect your running net payoff, or draw again; stopping before the first draw is also allowed. Find the maximal *expected additional profit* under an optimal stopping rule, including the value when the bag initially contains 26 balls of each color. The running net payoff may have become negative; the option to stop still adds zero *future* profit.

The recognition clue is the changing conditional probability: when r red and b black balls remain, the next draw is red with probability $r/(r+b)$, not necessarily the original $R/(R+B)$. The precise order of earlier draws is irrelevant once these two remaining counts are known, so (r, b) is a sufficient state.

Tiny-state calculation and two ways to define value

Begin with $V(1,0) = 1$, since the only remaining ball is red, and $V(0,1) = 0$, since it is better to stop than draw the sole black ball. From $(1,1)$, stopping adds zero, while drawing red gives $1 + V(0,1) = 1$ with probability $1/2$, and drawing black gives $-1+V(1,0) = 0$ with probability $1/2$. Thus $V(1,1) = 1/2$: an ability to stop can have strictly positive value even when equal numbers of red and black balls remain. Similarly, $V(2,1) = \frac{2}{3}(1 + V(1,1)) + \frac{1}{3}(-1 + V(2,0)) = 4/3$.

A two-dimensional table can be filled in increasing r and b, because every draw reduces $r + b$ by exactly one: $V(r,b)$ reads only $V(r-1,b)$ and $V(r,b-1)$, which are already known. Here V is *future incremental profit*, so each transition adds $+1$ or -1. If instead $F(r,b)$ means the *final total wealth including historical winnings*, the stop branch must include the existing payoff, and the same gains must not be added again. The full table makes this bookkeeping easier to audit.

Problem: A bag contains R red balls and B black balls. You draw uniformly without replacement. A red draw earns $+1$ dollar; a black draw loses -1 dollar. You may stop whenever you wish, including before the first draw. Find the maximum *expected incremental profit*.

State: $V(r,b)$ is the optimal expected profit that can be **earned from this point onward** with r red and b black balls remaining. Thus stopping now pays 0. The probabilities of red and black are $r/(r+b)$ and $b/(r+b)$. **Base**: $V(r,0) = r$ (draw all the

remaining red balls), $V(0,b) = 0$ (stop if only black remain), and $V(0,0) = 0$.

$$\boxed{V(r,b) = \max\left\{0, \frac{r}{r+b}[1 + V(r-1,b)] + \frac{b}{r+b}[-1 + V(r,b-1)]\right\}.}$$

Hand calculation: At $(1,1)$, continuation value is $\frac{1}{2}(1+0) + \frac{1}{2}(-1+1) = 0.5$, so $V(1,1) = 0.5$. For the corresponding standard 26-red/26-black card game, the initial value is approximately 2.62448.

```python
def red_black(R: int, B: int) -> float:
    dp = [[0.0] * (B + 1) for _ in range(R + 1)]
    for r in range(1, R + 1):
        dp[r][0] = float(r)

    for r in range(1, R + 1):
        for b in range(1, B + 1):
            continue_ev = (
                r / (r + b) * (1 + dp[r - 1][b])
                + b / (r + b) * (-1 + dp[r][b - 1])
            )
            dp[r][b] = max(0.0, continue_ev)
    return dp[R][B]

assert abs(red_black(1, 1) - .5) < 1e-12
assert round(red_black(26, 26), 2) == 2.62
```

Complexity: $O(RB)$ time and $O(RB)$ space. **Space optimization:** Each update reads $dp[r-1][b]$ from the previous row and the already-computed $dp[r][b-1]$ from the current row. With r outermost and b increasing, the two-dimensional table can be compressed into an array of length $B+1$:

```python
def red_black_rolling(R: int, B: int) -> float:
    dp = [0.0] * (B + 1)  # previous red-count row
    for r in range(1, R + 1):
        dp[0] = float(r)
        for b in range(1, B + 1):
            cont = (r * (1 + dp[b])
                    + b * (-1 + dp[b - 1])) / (r + b)
            dp[b] = max(0.0, cont)
    return dp[B]
```

Complexity: $O(RB)$ time, $O(B)$ space. *Why it works:* Before updating $dp[b]$, it still holds the previous row's $V(r-1,b)$, while $dp[b-1]$ already holds the current row's $V(r,b-1)$.

Common Pitfall

Alternatively define $F(r,b)$ as the **optimal final total payoff including previously accumulated winnings**. For equal starting red and black counts, accumulated winnings can be inferred as $b-r$, so stopping pays $b-r$, and transitions no longer add ± 1 separately. Both definitions are valid: $F(r,b) = (b-r) + V(r,b)$, but they must not be mixed.

2.5 Canonical Problem 3: An Infinite-Horizon Die Game with Ruin on Six

Full problem statement: Gambling with no fixed number of turns

Start with \$0 and repeatedly roll a fair six-sided die. A result of 1 through 5 adds the rolled number of dollars to your cumulative bankroll. A 6 confiscates *all* accumulated winnings and ends the game immediately. After any safe result, you may cash out your entire bankroll or risk another roll; there is no maximum number of rolls. Find both the *globally optimal stopping rule* and the risk-neutral entry value when starting at zero. The question is more demanding than whether merely one further roll has positive expected gain.

The sufficient state is current bankroll n: reaching \$10 through $5+2+3$ or $4+4+2$ gives exactly the same future distribution under these rules. Memorizing the order of previous safe rolls provides no additional predictive information.

Risk decomposition, optimal threshold, and backward order

If you roll once more and immediately stop after any safe roll, the expected *increment* is

$$\frac{1+2+3+4+5}{6} - \frac{n}{6} = 2.5 - \frac{n}{6}.$$

The first term is expected new earnings from safe outcomes; the second is the expected loss of the existing bankroll upon ruin. The one-step break-even amount is $n = 15$. For $n < 15$, taking one more roll and then stopping already beats stopping immediately. For $n \geq 15$, any safe result moves to an even higher bankroll, still in the region where one-step conditional expected gain is nonpositive. A suitable nonnegative supermartingale/optional-stopping argument establishes that stopping is globally optimal throughout that region.

Begin backward from $V(15) = 15, \ldots, V(19) = 19$. Then $V(14) = 85/6 \simeq 14.1667$ and $V(13) = (V(14)+15+16+17+18)/6 \simeq 13.3611$. Continue downward to obtain $V(0) \simeq 6.15374$. In the code, `risky_dice_value(n)` returns the optimal expected *final total bankroll*, so its transition uses $V(n+j)$ without adding j again. For a future-*increment* value function, explicitly include each new gain and potential loss in the recurrence.

Problem: On each roll, a result from 1–5 adds that many dollars to your total; a 6 erases all accumulated winnings and ends the game immediately. You may stop after any safe roll. Find the optimal stopping rule and initial fair value under risk neutrality.

State: Accumulated wealth n. The value of **rolling exactly once more and then stopping** is

$$C_1(n) = \frac{1}{6}[(n+1) + \cdots + (n+5) + 0] = \frac{5}{6}n + 2.5.$$

Comparing this with stopping at n, $C_1(n) > n \iff n < 15$. When $n \geq 15$, the conditional one-step expected gain is nonpositive, and every safe result keeps the process in the $n \geq 15$ region; a supermartingale/stopping argument therefore rules out a better continuation strategy. When $n < 15$, even rolling only once more and then stopping

strictly improves expected value.

$$V(n) = \begin{cases} n, & n \geq 15, \\ \frac{1}{6}\sum_{j=1}^{5} V(n+j), & n < 15. \end{cases}$$

At $n = 15$, stopping ties with "roll once more, then stop" in expectation; we choose to stop. Compute $V(14) = (15{+}16{+}17{+}18{+}19)/6 = 85/6$, and work backward to $V(0) \approx 6.15374$.

```python
from functools import cache

@cache
def risky_dice_value(n: int = 0) -> float:
    if n >= 15:
        return float(n)
    return sum(risky_dice_value(n + j) for j in range(1, 6)) / 6

assert round(risky_dice_value(), 2) == 6.15
```

Advanced interview point: In general, *one-step lookahead does not guarantee* a globally optimal policy. Explain why the stopping region is preserved by safe transitions and why continued play cannot improve expected wealth. If ruin probabilities, rewards, or cash-out rules change, solve the new Bellman equation instead of blindly reusing the threshold 15.

2.6 Canonical Problem 4: Replicating a Series Payoff with Bets (Binomial Hedging)

Full problem statement: Exact replication, not forecasting win probabilities

Two teams play a best-of-seven series: the first to win four games takes the championship. You may wager an amount y on the Red Sox in each individual game. A win adds y to your net profit; a loss subtracts y. Odds are symmetric and there are no fees. You cannot simply purchase a contract on the overall championship: you must adjust your wager *after each game*. Design a strategy that delivers a total net payoff of *exactly* +\$100 if the Red Sox win the championship and −\$100 otherwise, regardless of the intermediate sequence of outcomes. Determine the wager at each current series score (i, j) and verify that the replication starts from a net profit of zero at $0:0$.

Two backward steps and the binomial-hedging interpretation

At $3:3$, a single game remains. The terminal outcomes are $+100$ or -100, so $f(3,3)+y = 100$ and $f(3,3) - y = -100$, giving $f(3,3) = 0$ and $y = 100$. At $3:2$, a Red Sox win immediately produces $f(4,2) = 100$, while a loss leads to the known $f(3,3) = 0$. Thus $f(3,2) = (100+0)/2 = 50$ and $y(3,2) = (100-0)/2 = 50$. The weight $1/2$ is derived from the two symmetric payoff-replication equations; it is *not* an assumption about real-world baseball win probabilities.

Executable two-dimensional backward-induction code:

```python
def series_hedge(target=100):
```

```
    f = [[0.0] * 5 for _ in range(5)]
    bets = [[None] * 4 for _ in range(4)]
    for j in range(4):
        f[4][j] = float(target)
    for i in range(4):
        f[i][4] = float(-target)
    for i in range(3, -1, -1):
        for j in range(3, -1, -1):
            win, lose = f[i + 1][j], f[i][j + 1]
            f[i][j] = (win + lose) / 2
            bets[i][j] = (win - lose) / 2
    return f[0][0], bets

initial, bets = series_hedge()
assert initial == 0 and bets[3][3] == 100
assert bets[3][2] == 50 and bets[0][0] == 31.25
```

Only the two win counts (i, j) affect the future. The fixed seven-game problem has constant state count; for a first-to-K-wins series, time and space are $O(K^2)$. To validate replication, enumerate any feasible sequence of wins and losses, add the wagers' realized profits, and verify the final result is always exactly $+100$ or -100.

Problem: In a best-of-seven series (first to four wins), you may place an even-money 1:1 bet on the Red Sox before each game. Regardless of the path, your final net payoff must be exactly $+100$ if they win the championship, and exactly -100 otherwise. Find the stake at every possible score. Let $f(i, j)$ denote the *required cumulative net payoff* when the teams have won i and j games. Terminal conditions are $f(4, j) = 100$ and $f(i, 4) = -100$. After staking $y(i, j)$, both outcome branches must satisfy

$$f(i, j) + y = f(i + 1, j), \quad f(i, j) - y = f(i, j + 1).$$

Solving gives

$$\boxed{f(i, j) = \frac{f(i + 1, j) + f(i, j + 1)}{2}}, \qquad \boxed{y(i, j) = \frac{f(i + 1, j) - f(i, j + 1)}{2}}.$$

For example, $f(3, 3) = 0$, so bet 100 in the final game; $f(3, 2) = 50$, so bet 50 next. Initially $f(0, 0) = 0$. This is not about maximizing expected winnings under actual championship probabilities. It is a **pathwise replication of terminal payoffs** by dynamic hedging; the $1/2$ in the recurrence follows from symmetric betting gains and losses, not from the teams' actual win probabilities.

2.7 Algorithm Selection and Checks for This Family

- With a fixed terminal stage, prioritize backward induction; a small state space can use a simple two-dimensional table.
- With multiple random outcomes, specify the probability and **immediate reward** of each transition before taking an expectation.
- For stop/continue choices, make sure the maximum lies outside the expectation over the next random result; when a new decision can be made after observing that result, the value V at the next state already contains that next maximum.

- For infinite horizons, discuss thresholds, convergence, or absorption; `@cache` alone does not ensure recursion terminates.

3 Family Two: Sequential Decisions and State Machines

3.1 Structure and Recognition: How Historical Rules Change Your Current Status

Clues: Progressing by day or minute, buying or selling today, holding versus cash, at most K trades, cooldown periods, limits on repeated actions, and fees. The structure is often a **time axis t plus a small finite set of states**. If future legal actions depend on your prior "status," that status belongs in the state.

Core technique: Draw the graph of legal state transitions: *which old states can lead into each new state?* Assign the immediate reward or cost to each edge and take the maximum. With a constant number of statuses, $O(n)$ time and $O(1)$ auxiliary space are often possible.

3.2 Recognition in Detail: When Must We Remember Yesterday's Status?

This model **processes a time sequence or an array one element at a time, while the currently legal actions depend on recent history**. The question need not involve stocks; consider a state machine whenever it has one of these rules:

1. **Cannot select adjacent positions:** "Bonuses are arranged in a row; selecting position i forbids selecting $i+1$." This local conflict allows a prefix-optimal take/skip recurrence, without necessarily requiring an explicit position label.
2. **Holding versus being in cash:** "Open, close, or maintain a position each day, but never hold two long positions simultaneously." The day alone is insufficient: if you already hold a share, you cannot buy another without restriction. The state must record *whether a position is held at day's end*.
3. **One-day cooldown after selling:** "If you sold yesterday, you may not buy today." The nonholding state splits into *just sold (cannot buy immediately)* and *resting in cash (can buy)*. Otherwise the model incorrectly allows repurchasing the next day.
4. **Limit on completed trades:** "At most two complete buy-sell transactions." Record the number completed or remaining; two paths arriving on the same day with the same position can have different feasible future actions if their trade counts differ.
5. **Limit on consecutive actions:** "You cannot go long three days consecutively" or "quotes may deviate in the same direction for at most two rounds." Store the *previous action and its streak length*; transitions breaking the restriction are invalid.

Counterexample: If the task merely asks for the maximum among independent

numbers with no temporal coupling or restrictions, a single scan suffices. Do not infer DP from the words "maximum profit." Find *how today's decision restricts tomorrow's legal actions*.

3.2.1 Drawing a State Machine by Hand on a Tiny Example

Suppose daily prices are $[1, 3, 2]$ and one transaction is allowed. At the end of day 1, there are two distinct situations: *cash with profit 0*, or *holding one share with net cash flow* -1. On day 2, with price 3, a holder can sell and enter the cash state with profit $-1 + 3 = 2$; someone in cash can buy and enter the holding state with cash balance -3. These cannot be combined into a single "maximum profit on day 2," because their future legal actions are different.

For a multistate DP, first list each legal edge:

$$\begin{aligned}
\text{Cash}_{t-1} &\longrightarrow \text{Cash}_t \quad \text{(do nothing, reward 0)},\\
\text{Cash}_{t-1} &\longrightarrow \text{Hold}_t \quad \text{(buy, cash flow } -p_t\text{)},\\
\text{Hold}_{t-1} &\longrightarrow \text{Hold}_t \quad \text{(continue holding)},\\
\text{Hold}_{t-1} &\longrightarrow \text{Cash}_t \quad \text{(sell, cash flow } +p_t\text{)}.
\end{aligned}$$

For each *new state*, enumerate the *old states* that can enter it; add the edge reward and maximize. **All new states on a given day should transition from the previous day's states**. Initially use `new = old.copy()` rather than rushing into in-place compression.

3.2.2 Do Not Mix Up Two Definitions of Subproblems

Forward definition: $dp[t][s]$ is the maximum accumulated profit *after processing day* t *and ending with status* s; convenient for bottom-up. **Backward definition:** `dfs(t,s)` is the maximum additional future profit *starting on day* t *with status* s; convenient for top-down. They may be equivalent, but account for payoffs at different points in time. Specify whether the value already includes historical purchase costs so you do not deduct a buying cost twice.

3.2.3 Manual Example: Why a Cooldown Splits the State

Consider prices $[1, 2, 3, 0, 2]$. Without a cooldown, buy at 1 and sell at 3 for 2, then buy at 0 and sell at 2 for another 2: total profit 4. But buying at 0 on the day immediately after selling at 3 violates the cooldown. Instead, buy at 1 on day 1, sell at 2 on day 2 (profit 1), rest on day 3, buy at 0 on day 4, and sell at 2 on day 5 (profit 2): total 3 (days are numbered starting from 1 here). To prevent illegal immediate rebuying, the program must distinguish the two nonholding states `sold` and `rest`.

3.3 Canonical Problem 1: House Robber — No Adjacent Selections

Full problem statement: Why choosing one item excludes its neighbor

You are given n houses in a row; house i contains a nonnegative amount a_i. A security system triggers if two adjacent houses are robbed, so no two selected indices may

be consecutive. You may select no houses. Return the maximum sum of money over all legal subsets, not necessarily the list of selected indices. For example, for $[2,7,9,3,1]$, choosing houses $0,2,4$ yields 12, whereas choosing $1,3$ yields 10, so the answer is 12. Greedily choosing the largest individual house can prevent a better total combination.

Write a complete prefix table before compressing space

An intuitive forward definition is $dp[i]$: maximum profit using only the *first i houses*, so the i-th considered house is a_{i-1}. Skipping it gives $dp[i-1]$; taking it excludes its predecessor and gives $dp[i-2]+a_{i-1}$. Use $dp[0]=0,\ dp[1]=a_0$ (handle an empty array separately). For $[2,7,9,3,1]$ the entire table is $[0,2,7,11,11,12]$.

```python
def rob_table(nums):
    n = len(nums)
    if n == 0:
        return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n + 1):
        dp[i] = max(dp[i - 1], dp[i - 2] + nums[i - 1])
    return dp[n]
```

This prefix-based formulation and the existing `dfs(i)` formulation solve the same problem in opposite directions: the former optimizes a processed prefix, the latter optimizes the remaining suffix. Only after seeing that each entry depends on the previous two should you compress the table to two rolling variables. If amounts may be negative, clarify whether selecting nothing is legal; when it is, negative houses can simply be skipped.

Problem: Given an array of nonnegative numbers, choose nonadjacent positions to maximize their sum. For $[2,7,9,3,1]$, the answer is 12 (positions 0, 2, and 4).

State: $F(i)$ is the maximum reward obtainable from index i onward. **Transition**: Skipping i gives $F(i+1)$; taking i gives $a_i + F(i+2)$:

$$F(i) = \max\{F(i+1), a_i + F(i+2)\}, \qquad F(i) = 0 \ (i \geq n).$$

Top-Down (the most natural derivation):

```python
from functools import cache

def rob_topdown(nums):
    n = len(nums)
    @cache
    def dfs(i):
        if i >= n:
            return 0
        return max(dfs(i + 1), nums[i] + dfs(i + 2))
    return dfs(0)
```

Bottom-Up ($O(1)$ space):

```python
def rob(nums):
```

```
    prev2 = prev1 = 0
    for x in nums:
        prev2, prev1 = prev1, max(prev1, prev2 + x)
    return prev1
```

Complexity: $O(n)$ time; memoization uses $O(n)$ cache and stack space; rolling states use $O(1)$ auxiliary space. **Variations:** If houses form a circle (House Robber II), the first and last cannot both be taken; solve the two linear cases excluding the first or excluding the last, and take the maximum (handle length 1 separately). If the houses form a tree, return *take/skip* values per node, giving Tree DP.

3.4 Canonical Problem 2: Kadane's Algorithm — Maximum Subarray Sum

Full problem statement: Contiguous and nonempty are hard constraints

Given an integer array containing positive and negative numbers, select a *contiguous, nonempty* interval $[l, r]$ with maximum sum. You may not skip undesirable elements within the selected interval. In $[-2, 1, -3, 4, -1, 2, 1, -5, 4]$, the best subarray is $[4, -1, 2, 1]$ with sum 6. If every element is negative, such as $[-5, -2, -9]$, the correct answer is -2, not 0: choosing an empty subarray is forbidden.

From enumerating intervals to a state that must end at i

Brute force considers $O(n^2)$ pairs of endpoints; prefix sums can evaluate each interval quickly, but do not reduce the number of candidates. The real state reduction comes from fixing the right endpoint. The best subarray that *must end at* i either starts afresh with a_i or extends the best subarray ending at $i-1$. Therefore $E_i = \max(a_i, E_{i-1} + a_i)$. Along $[4, -1, 2, 1]$, the best ending sums are $4, 3, 5, 6$. The global answer is the maximum over *all* endpoints, not necessarily the final DP entry.

```
def kadane_table(a):
    dp = [0] * len(a)
    dp[0] = a[0]
    for i in range(1, len(a)):
        dp[i] = max(a[i], dp[i - 1] + a[i])
    return max(dp)
```

The rolling-variable implementation simply replaces $dp[i-1]$ by `end`. If the interviewer also asks for the interval itself, reset the candidate left endpoint when you start anew at a_i, and record both endpoints whenever you improve the global best.

Problem: From an array that may contain positive and negative numbers, choose a *contiguous, nonempty* interval with maximum sum. For $[-2, 1, -3, 4, -1, 2, 1, -5, 4]$, the answer is 6, from $[4, -1, 2, 1]$.

State: $E(i)$ is the maximum contiguous-subarray sum *that must end at index* i. Either start anew at a_i, or extend the preceding interval:

$$E(i) = \max\{a_i, E(i-1) + a_i\}, \quad E(0) = a_0, \quad \text{answer} = \max_i E(i).$$

```python
def max_subarray(nums):
    end = ans = nums[0]
    for x in nums[1:]:
        end = max(x, end + x)
        ans = max(ans, end)
    return ans
```

Complexity: $O(n)$ time, $O(1)$ space. **Pitfalls:** Do not return 0 for an all-negative array (the interval must be nonempty), and do not simply return the last $E(n-1)$. You may also track the start of each newly initiated interval and the best end point to recover the subarray.

House Robber versus Kadane: Why Are These Two One-Dimensional Maxima Different?

House Robber's $F(i)$ is the maximum payoff from position i onward, *with skipping allowed*; its two branches are **skip the current element** and **take the current element**. Kadane's $E(i)$, however, *must end at element* i; its two branches are **start a new contiguous interval at** i or **extend the immediately preceding interval**. For $[4,-1,2,1]$: $E(0)=4$; $E(1)=\max(-1,4-1)=3$; $E(2)=\max(2,3+2)=5$; $E(3)=\max(1,5+1)=6$. Each step corresponds to a natural binary choice.

This is why trading and time-series problems must distinguish *being allowed to skip dates* from *being required to select one contiguous interval*. Maximum profit from a continuous holding period is not the same model as choosing nonadjacent trading opportunities.

3.5 Canonical Problem 3: Maximum Product Subarray — Track the Minimum Too

Full problem statement: Why negative numbers defeat a single maximum state

Find the maximum product of a *contiguous, nonempty* subarray. For $[2,3,-2,4]$, the answer is 6 from $[2,3]$, whereas for $[-2,3,-4]$ the answer is 24 from the entire array. Unlike addition, multiplication by a negative number reverses order: a previously terrible negative product may become an excellent positive one. Zero resets products across its position and should allow the next subarray to start afresh.

First construct two full state arrays, then use two variables

At each index i, save both M_i, the largest product of a subarray ending at i, and m_i, its smallest product. For new element x, the valid candidates are x alone, xM_{i-1}, and xm_{i-1}. Compute the new maximum and minimum from the *same three candidates*. In $[-2,3,-4]$, both values start at -2; after 3 they are 3 and -6; after -4, the candidate values $-4,-12,24$ give a new maximum of 24 and minimum of -12.

```python
def max_product_table(nums):
    n = len(nums)
    hi = [0] * n
```

```
    lo = [0] * n
    hi[0] = lo[0] = nums[0]
    for i in range(1, n):
        x = nums[i]
        options = (x, x * hi[i - 1], x * lo[i - 1])
        hi[i] = max(options)
        lo[i] = min(options)
    return max(hi)
```

The two-array version uses $O(n)$ auxiliary space and is easier for checking that old values are not overwritten too early. Once the dependence is clear, update two temporary variables together to obtain $O(1)$ auxiliary space.

Problem: Find the largest product of a contiguous nonempty subarray. For $[-2, 3, -4]$, the answer is 24. Multiplication by a negative number reverses order, so keeping only the previous maximum is insufficient.

$$M_i = \max\{a_i, a_i M_{i-1}, a_i m_{i-1}\},$$
$$m_i = \min\{a_i, a_i M_{i-1}, a_i m_{i-1}\}.$$

Here M_i and m_i are the maximum and minimum product of a subarray that must end at i. Initialize both to a_0 and return $\max_i M_i$.

```
def max_product(nums):
    mx = mn = ans = nums[0]
    for x in nums[1:]:
        a, b, c = x, mx * x, mn * x
        mx, mn = max(a, b, c), min(a, b, c)
        ans = max(ans, mx)
    return ans
```

Complexity: $O(n)$ time, $O(1)$ space. **Key idea:** A currently unfavorable negative value may become optimal after a future negative multiplier. If future transitions depend on several *extreme-value directions*, preserve enough states for all of them.

3.6 Canonical Problem 4: At Most K Completed Stock Trades

Full problem statement: Transactions are completed on selling, not buying

Given daily stock prices `prices`, you start with zero cash profit and no position. You may execute at most K complete buy-then-sell transactions. Each transaction buys one share and sells it on a later date; you can hold at most one share, cannot short, and must be flat at the end. Making no trades is allowed. For `prices=[3,2,6,5,0,3]`, $K = 2$, buy at 2 and sell at 6 (profit 4), then buy at 0 and sell at 3 (profit 3), for a total of 7. You must remember both today's position status and the number of completed *sales*, not merely the day index.

Why cash and holding need different states

Begin with a full three-dimensional conceptual table $D[t][j][h]$: first t days processed; *exactly* j completed sales; $h \in \{0, 1\}$ meaning flat or holding a share. A holding state's

recorded profit is the cash account after the purchase, not the wealth after marking the unsold position to market. At a new price p, a cash state may be carried forward or reached by selling yesterday's position (one more completed sale). A holding state may be carried forward or reached by buying from yesterday's cash state.

```python
def max_profit_k_table(prices, K):
    n = len(prices)
    NEG = float('-inf')
    dp = [[[NEG, NEG] for _ in range(K + 1)]
          for _ in range(n + 1)]
    dp[0][0][0] = 0
    for t, p in enumerate(prices, 1):
        for j in range(K + 1):
            dp[t][j][0] = max(dp[t][j][0], dp[t-1][j][0])
            dp[t][j][1] = max(dp[t][j][1], dp[t-1][j][1])
            dp[t][j][1] = max(dp[t][j][1], dp[t-1][j][0]-p)
            if j:
                dp[t][j][0] = max(dp[t][j][0], dp[t-1][j-1][1]+p)
    return max(dp[n][j][0] for j in range(K + 1))
```

This full table takes $O(nK)$ space. The familiar `cash/hold` arrays drop the time dimension to use $O(K)$ space. Read all inputs from *yesterday's* states when constructing today's states: mixing newly updated and old values can accidentally allow trades on the same day that were not part of the model.

Problem: Given daily prices $p_0, \ldots, p_{n-1}$, make at most K complete *buy-then-sell* transactions, holding at most one share at a time. Maximize profit and finish with no open long position. Doing nothing is allowed.

State: $cash_t[j]$ is the maximum accumulated cash P&L after day t while *flat and having completed exactly j sales*. $hold_t[j]$ is the maximum cash P&L while *holding a share and having completed exactly j sales*. Buying does not increment the completed count; selling completes one trade.

$$newHold[j] = \max(oldHold[j], oldCash[j] - p),$$

$$newCash[j] = \max(oldCash[j], oldHold[j-1] + p) \quad (j \geq 1).$$

Base: $cash[0] = 0$; all other $cash[j]$ and all $hold[j]$ start at $-\infty$. The answer is $\max_{0 \leq j \leq K} cash[j]$. Updating each day simultaneously from *old states* is the clearest approach and prevents unwanted same-day chaining.

```python
def max_profit_k(prices, K):
    neg = float('-inf')
    cash = [0] + [neg] * K
    hold = [neg] * (K + 1)
    for p in prices:
        new_cash, new_hold = cash[:], hold[:]
        for j in range(K + 1):
            new_hold[j] = max(hold[j], cash[j] - p)
            if j >= 1:
                new_cash[j] = max(cash[j], hold[j - 1] + p)
        cash, hold = new_cash, new_hold
    return max(cash)
```

Example: Prices $[3, 2, 6, 5, 0, 3]$, $K = 2$. Buy at 2 and sell at 6; buy at 0 and sell at 3. Profit $4 + 3 = 7$. **Complexity**: $O(nK)$ time and $O(K)$ space. If $K \geq \lfloor n/2 \rfloor$, the constraint becomes nonbinding; sum all positive consecutive price changes in $O(n)$ time and $O(1)$ space.

Common Pitfall

Do not confuse "exactly j completed sales" with "at most j trades." Nor should you initialize every $cash[j]$ to 0 while claiming it represents *exactly j completed sales*. Incorrect in-place update order can contaminate same-day states. Start with separate old/new arrays in interviews, then optimize.

Tracing Three Crucial Cases for At Most K Transactions

Take prices $[3, 2, 6, 5, 0, 3]$ with at most two completed trades. The first profitable peak is 6: buy at 2 and sell at 6 for 4; the second is 3: buy at 0 and sell at 3 for 3. Combined profit is 7. You cannot treat the same-day sale at 6 as a fresh purchase at 6 if the model requires valid temporal sequencing of distinct trades. The "completed sales" counter determines which $cash[j]$ is available for the next purchase.

If $K = 0$, the answer must be 0; if prices decrease strictly, not trading is optimal, also earning 0; if K is large enough to trade on every uptrend, the answer reduces to the sum of all positive day-to-day price changes. Mentioning these three boundary cases helps demonstrate understanding of the state's economic meaning rather than mere memorization of code.

3.7 Canonical Problem 5: Stock Trading with a Cooldown

Full problem statement: A sale blocks the next day's purchase

Given daily stock prices, you can buy and sell repeatedly, while holding at most one share and never shorting. After selling, you are forbidden to buy throughout the *next full trading day* (a one-day cooldown). Maximize profit, finishing with no shares. For $[1, 2, 3, 0, 2]$, buy at 1, sell at 2, rest the next day, then buy at 0 and sell at 2, realizing total profit 3. Ignoring the cooldown would combine incompatible trades.

A day-by-day table distinguishes available cash from forced rest

If we record only `hold` and `cash`, today's cash could have resulted from a sale today or from being out of the market for several days; those states have different permissions for the next day's purchase. The minimum useful set of statuses is `hold` (end the day with a share), `sold` (sold today), and `rest` (flat, and did not sell today). A new `hold` can come from yesterday's `hold` or `rest`, *never* from yesterday's `sold`. A new `sold` comes from yesterday's `hold` plus today's sale price, and a new `rest` comes from yesterday's `rest` or `sold`.

```
def cooldown_table(prices):
    n = len(prices)
    NEG = float('-inf')
    dp = [[NEG, NEG, NEG] for _ in range(n + 1)]
```

```
    # indexes: hold=0, sold=1, rest=2
    dp[0][2] = 0
    for t, p in enumerate(prices, 1):
        h, s, r = dp[t - 1]
        dp[t] = [max(h, r-p), h+p, max(r, s)]
    return max(dp[n][1], dp[n][2])
```

The complete time-by-status table uses $O(n)$ time and space; once it is clear that only yesterday matters, the three statuses can be rolled into $O(1)$ space. A sale fee, if added, belongs on the transition `hold -> sold` rather than on every possible edge.

Problem: You may buy and sell any number of times, but you cannot buy on the day immediately after selling. Hold at most one share. For $[1,2,3,0,2]$, maximum profit is 3. **Three statuses:** *hold* (holding a share), *sold* (just sold today), and *rest* (flat and eligible to buy). At daily price p:

$$newHold = \max(hold, rest - p),$$

$$newSold = hold + p,$$

$$newRest = \max(rest, sold).$$

Base: $hold = -\infty, sold = -\infty, rest = 0$; the answer is $\max(rest, sold)$.

```
def max_profit_cooldown(prices):
    hold = sold = float('-inf')
    rest = 0
    for p in prices:
        hold, sold, rest = (
            max(hold, rest - p),
            hold + p,
            max(rest, sold)
        )
    return max(rest, sold)
```

Complexity: $O(n)$ time and $O(1)$ space. **Transaction-fee variation:** Deduct fee on the selling edge; for example, without a cooldown:

$$newCash = \max(cash, hold + p - fee), \quad newHold = \max(hold, cash - p).$$

This illustrates the **general state-machine DP method**: draw legal transitions between statuses first, then enumerate all predecessor statuses for each new status, rather than guessing transitions from the code's loop structure.

3.8 Advanced Variations: Additional States and Rolling Optimization

If the task limits consecutive holding days, completed trades, or previous actions, add the corresponding labels to the state, such as (t, s, k, c). When k and c have small ranges, complexity is often $O(nKS)$; if transitions depend only on the preceding time layer, roll two dictionaries or two two-dimensional arrays. **Real-world price predictions, fill probabilities, and impact costs** in backtests must be explicit model assumptions. State-machine DP optimizes the specified model; it does not guarantee genuine trading alpha.

4 Family Three: Resource Allocation and Knapsack

4.1 Recognition: Take or Skip, and How Does Capacity Change?

Clues: Limits on weight or capital, using each item once, reaching an exact monetary target, counting combinations, and multiple resource budgets. First determine the allowed number of uses (0/1, unlimited, or bounded); next identify the objective (maximize, minimize, feasibility, or counting); finally distinguish *at most* a capacity from *exactly* filling it.

$$0/1\text{:}\ dp_i[c] = \max\{dp_{i-1}[c], dp_{i-1}[c-w_i] + v_i\};$$

$$\text{Unbounded:}\ dp_i[c] = \max\{dp_{i-1}[c], dp_i[c-w_i] + v_i\}.$$

The difference is whether the take branch can still reuse the current item. With *items as the outer loop and in-place one-dimensional updates*, scan capacities **downward** for 0/1 and **upward** for unbounded knapsack. Descending reads the old state before it is updated in this item iteration; ascending allows reuse of freshly updated states.

4.2 Recognition in Detail: Knapsack Is Constrained Selection, Not Literally Packing a Bag

Knapsack questions can be phrased using investments, coins, inventory, or target amounts. Ask: *Are there objects consuming resources, and must I choose among them under a total resource constraint?*

1. **Discrete choices with maximum reward:** "You can invest at most 8 budget units; each project requires a fixed amount, must be undertaken completely or not at all, and earns a payoff." This is standard 0/1 knapsack, with $dp[i][c]$ and $\max(\text{skip}, \text{take})$.

2. **Exact target:** "Given some numbers, can you select a subset summing to exactly T?" This is Subset Sum. The objective is *reachability* rather than maximum value, so use Boolean DP.

3. **Reusable resource types:** "Unlimited coins of denominations 1, 3, and 4 are available; what is the fewest coins needed for 6?" This is unbounded knapsack: *the same denomination can be used repeatedly*, unlike choosing a specific array element only once.

4. **Count rather than check existence:** "How many distinct combinations make the amount?" The same $dp[c]$ can change from a Boolean to an integer count, but clarify *whether different orders count as different solutions*.

5. **Limited inventory or several budgets:** "At most three of each component" gives bounded knapsack; "choose at most one project per category" gives group knapsack; simultaneous capital and labor limits give two-dimensional knapsack.

Decisive wording: "Each element at most once" means 0/1, "unlimited copies of each denomination" means unbounded, and "at most k copies" means bounded. Do not be distracted by item names. The **objective** determines the operator: max for greatest value, min for least count, OR for feasibility, and addition for counting.

4.2.1 Start with a Two-Dimensional Table to Get the Intuition Right

State $dp[i][c]$ is the maximum value using the first i items with capacity at most c. When processing the ith item, skipping uses $dp[i-1][c]$; taking uses $dp[i-1][c-w_i]$ from the preceding item layer. This **automatically ensures that the current item is used at most once**. The simplest implementation should use a two-dimensional table first, even if its space usage is larger, because it is easier to explain and verify.

4.2.2 Why Does One-Dimensional Compression Change the Capacity Order?

After removing dimension i, reading `dp[c-w]` may retrieve either the *old value* before processing this item or the *new value* already updated during the current item. A 0/1 item must read the old value, so iterate from W down to w. An unbounded item can be taken repeatedly, so iterate from w up to W, letting `dp[c-w]` include the current item already selected.

Minimal counterexample: one item of weight 2 and value 3, with capacity 4. Scanning upward incorrectly yields $dp[2] = 3$ and then $dp[4] = dp[2]+3 = 6$, effectively using the same item twice. When scanning downward, $dp[2]$ is still its old value 0 at the moment $dp[4]$ is computed, so the result is 3.

4.2.3 Initialization Comes from Problem Semantics, Not Memorized Code

Objective	Typical initialization	Reason
Maximum value with capacity at most W	All zeros	Selecting nothing is always feasible
Minimum count to reach W exactly	$dp[0] = 0$; others $+\infty$	Amounts without a constructed solution are unreachable
Maximum value to reach W exactly	$dp[0] = 0$; others $-\infty$	An empty selection must not pretend to fill the bag
Feasibility	$dp[0] = True$; others False	Only sum 0 is initially known to be reachable
Combination count	$dp[0] = 1$; others 0	The empty combination is one construction

Combinations versus permutations, revisited: With coins 1 and 2 making amount 3, $1+2$ and $2+1$ are one solution when order is ignored, but two different ordered sequences. Classic unbounded combination counting puts *items in the outer loop*; ordered-sequence counting puts *amount in the outer loop and last coin in the*

inner loop. This is a separate issue from scanning capacities upward/downward to control item reuse.

4.3 Canonical Problem 1: Standard 0/1 Knapsack

Full problem statement: Capacity is a ceiling, not a target that must be filled

There are n indivisible items. Item i has positive integer weight w_i and value v_i, and only one copy is available. Either take the entire item or leave it. The total weight must be *at most* the knapsack capacity W; unused space is allowed. Maximize *total value*, not weight utilization. For `weights=[2,3,4,5]`, `values=[3,4,5,6]`, and $W = 8$, take the items of weights 3 and 5: total weight 8 and value 10. Choosing weights $2 + 5$ or $3 + 4$ instead yields only 9. Hence the answer is 10. The recognition clue is that each item can be selected *at most once* and each choice consumes a limited resource.

Deriving the full two-dimensional Bottom-Up table

The existing Top-Down `dfs(i, cap)` looks ahead from item i. For beginners, a prefix table is more transparent: define $dp[i][c]$ as the greatest value obtainable using only the *first i items*, with total weight *not exceeding* c. The first row $dp[0][c] = 0$ means no items are available. The first column $dp[i][0] = 0$ reflects zero capacity. For item i (Python index $i-1$), *skip* yields $dp[i-1][c]$, and, when $w_{i-1} \le c$, *take* yields $dp[i-1][c-w_{i-1}] + v_{i-1}$. Both expressions read the *previous item row*:

$$dp[i][c] = \begin{cases} dp[i-1][c], & c < w_{i-1}, \\ \max\{dp[i-1][c],\ dp[i-1][c-w_{i-1}] + v_{i-1}\}, & c \ge w_{i-1}. \end{cases}$$

Start with this complete two-dimensional implementation:

```python
def knapsack_2d(weights, values, W):
    n = len(weights)
    dp = [[0] * (W + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        w, v = weights[i - 1], values[i - 1]
        for c in range(W + 1):
            # skip: do not choose item i-1
            dp[i][c] = dp[i - 1][c]
            if c >= w:
                # take: only states from PREVIOUS row
                dp[i][c] = max(dp[i][c],
                               dp[i - 1][c - w] + v)
    return dp[n][W]

assert knapsack_2d([2,3,4,5], [3,4,5,6], 8) == 10
```

The hand-calculated table, with columns $c = 0, 1, \ldots, 8$ and one row per processed item, is:

	0	1	2	3	4	5	6	7	8
0 items	0	0	0	0	0	0	0	0	0
$A(2,3)$	0	0	3	3	3	3	3	3	3
$A,B(3,4)$	0	0	3	4	4	7	7	7	7
$A,B,C(4,5)$	0	0	3	4	5	7	8	9	9
$A,B,C,D(5,6)$	0	0	3	4	5	7	8	9	10

For example, $dp[4][8] = \max(dp[3][8], dp[3][3] + 6) = \max(9, 4 + 6) = 10$. **In the full two-dimensional 0/1 table, capacity can be traversed in either direction**: the current row always reads from the previous row. The loop direction does not control item reuse here.

Why reverse capacity traversal is required only after in-place compression

The precise two-row update is

$$dp_{\text{new}}[c] = \max\{dp_{\text{old}}[c],\ dp_{\text{old}}[c - w] + v\}.$$

If we overwrite a *single* array from small capacities upward, $dp[c - w]$ may already include this item's contribution from the present pass, effectively selecting it twice. Minimal counterexample: one item with weight 2 and value 3, and capacity 4. Updating $dp[2] = 3$ and then using this new $dp[2]$ to calculate $dp[4] = 6$ illegally takes the same item twice. Traversing capacities *downward* means $c - w < c$ has not yet been overwritten, so it still represents the previous item row. The existing one-dimensional implementation changes storage, not the underlying optimization problem.

The learning sequence is: `dfs(i,cap)` for Take/Skip; $dp[i][c]$ to see the previous-row dependence; then one descending capacity array to save space. Both versions take $O(nW)$ time; space falls from $O(nW)$ to $O(W)$. To reconstruct *which items* were selected, the full two-dimensional table also makes it easy to trace decisions backward from (n, W).

Problem: Item i has positive weight w_i and positive value v_i. Each item can be selected at most once and cannot be split. Total weight must not exceed capacity W. Maximize total value. Example: $w = [2, 3, 4, 5]$, $v = [3, 4, 5, 6]$, $W = 8$; select weights 3 and 5 for value 10.

State: $D(i, c)$ is the maximum additional value from item i onward with available capacity c. **Base**: $D(n, c) = 0$. **Transition**:

$$D(i,c) = \begin{cases} D(i+1,c), & w_i > c, \\ \max\{D(i+1,c),\ v_i + D(i+1,c-w_i)\}, & w_i \le c. \end{cases}$$

Top-Down—the easiest form to derive from take/skip:

```python
from functools import cache

def knapsack_topdown(weights, values, W):
    n = len(weights)
    @cache
```

```
    def dfs(i, cap):
        if i == n:
            return 0
        best = dfs(i + 1, cap)  # skip
        if weights[i] <= cap:
            best = max(best,
                       values[i] + dfs(i + 1, cap - weights[i]))
        return best
    return dfs(0, W)
```

Bottom-Up—two-dimensional table: $dp[i][c]$ is the optimal value using the first i items; both transitions read the previous row. This uses $O(nW)$ space. Compressing further gives the descending one-dimensional update:

```
def knapsack_01(weights, values, W):
    dp = [0] * (W + 1)
    for w, v in zip(weights, values):
        for cap in range(W, w - 1, -1):
            dp[cap] = max(dp[cap], dp[cap - w] + v)
    return dp[W]
```

Complexity: $O(nW)$ time, $O(W)$ rolling space (pseudo-polynomial: W is the *numeric value* of capacity). If the bag must be *filled exactly*, initialize $dp[0] = 0$ and all other entries to $-\infty$, returning infeasible when appropriate; do not retain the default all-zero initialization.

4.4 Canonical Problem 2: Partition Equal Subset Sum (LeetCode 416)

Full problem statement: Two equal groups mean one half-sum subset

Given an array of *positive integers*, divide *all indexed elements* into two disjoint subsets with the same total sum. The two subsets need not have the same number of elements. For $[1, 5, 11, 5]$, use $[11]$ against $[1, 5, 5]$, so return True. For $[1, 2, 3, 5]$, the total is 11 and no equal split is possible, so return False. The two 5s at different indices are distinct available items; a single indexed 5 may not be used twice.

Two-dimensional Boolean DP before one-dimensional compression

Compute the total S. If S is odd, return False. Otherwise the target is $T = S/2$, and it is sufficient to find a subset with sum *exactly* T: the remaining items then also sum to T. Define $dp[i][c]$ to mean that the first i items can form sum c. Set $dp[0][0]$ = True and $dp[0][c > 0]$ = False. The empty subset forms zero from any prefix. For new item $x = a_{i-1}$, either skip it (read $dp[i-1][c]$) or take it (read $dp[i-1][c-x]$):

$$dp[i][c] = dp[i-1][c] \ \vee \ \big(c \geq x \ \wedge \ dp[i-1][c-x]\big).$$

```
def can_partition_2d(nums):
    S = sum(nums)
    if S % 2:
```

```
        return False
    T = S // 2
    n = len(nums)
    dp = [[False] * (T + 1) for _ in range(n + 1)]
    dp[0][0] = True
    for i in range(1, n + 1):
        x = nums[i - 1]
        for c in range(T + 1):
            dp[i][c] = dp[i - 1][c]
            if c >= x:
                dp[i][c] |= dp[i - 1][c - x]
    return dp[n][T]
```

For $[1, 5, 11, 5]$, the target is 11. After the first 1, the reachable sums are $\{0, 1\}$. After the first 5, they include $\{0, 1, 5, 6\}$. After processing 11, taking *that single 11* reaches the target, so the result is True. As in 0/1 maximization, capacity may be visited in either direction in the *two-dimensional* version because each candidate reads the previous row. Only after compressing both rows into the same `dp[c]` array must capacity be traversed downward to avoid reusing the newly updated `dp[c-x]`.

Problem: Can an array of positive integers be partitioned into two subsets with equal sums? For $[1, 5, 11, 5]$ the answer is True; for $[1, 2, 3, 5]$ it is False.

Let the total be S. Return False immediately if S is odd; otherwise seek a subset totaling exactly $S/2$. **State**: $dp[c]$ indicates whether processed elements can produce sum c. Initialize $dp[0] = True$ and the rest False. Each element can be used only once, so scan downward:

$$dp[c] \leftarrow dp[c] \vee dp[c - x].$$

```
def can_partition(nums):
    total = sum(nums)
    if total % 2:
        return False
    target = total // 2
    dp = [False] * (target + 1)
    dp[0] = True
    for x in nums:
        for c in range(target, x - 1, -1):
            dp[c] = dp[c] or dp[c - x]
    return dp[target]
```

Complexity: $O(nS)$ time, $O(S)$ space (actual capacity is $S/2$). **Advanced bitset DP:** Use the bits of a Python big integer to represent all reachable sums, and inspect bit T:

```
def can_partition_bitset(nums):
    s = sum(nums)
    if s % 2:
        return False
    target = s // 2
    bits = 1
    for x in nums:
        bits |= bits << x
    return bool((bits >> target) & 1)
```

Big-integer shifts and OR operations run in optimized native code and can be much faster than nested Python loops. Their theoretical cost still depends on bit length; this is not a free $O(n)$ algorithm. If needed, truncate to the lowest $T+1$ bits to reduce memory.

Partition: Why Descending Order? A Single-Item Proof

Suppose the input contains only one element, 2, and the target is 4. The *correct answer is False*, because the single 2 can be used only once. If one-dimensional Boolean DP scans upward, it first sets $dp[2] = dp[0] = True$, then immediately sets $dp[4] = dp[2] = True$, effectively selecting the same 2 twice. That is the entire reason for descending order: computing target 4 must not read target 2 created by the same item.

The two-dimensional table avoids this problem because the take candidate always reads the previous row $dp[i-1][c-w_i]$. **Understand the previous row in two dimensions before memorizing descending order in one dimension.**

4.5 Canonical Problem 3: Coin Change (LeetCode 322)

Full problem statement: Unlimited coins, exact change required

You have a set of *distinct positive-denomination* coin types, with unlimited supplies of each type. Given a target amount A, choose as few coins as possible such that their denominations sum to *exactly* A; return -1 if this cannot be done. With denominations $[1, 2, 5]$ and amount 11, the optimum is $5+5+1$ using three coins. With $[2]$ and amount 3, no solution exists. Do *not* blindly choose the largest available denomination: for $[1, 3, 4]$ and target 6, greedy produces $4+1+1$ (three coins), while $3+3$ uses only two.

A full two-dimensional table shows why an unbounded item may repeat

Let $dp[i][a]$ be the fewest coins needed to make *exactly* amount a using only the first i coin denominations. Set $dp[0][0] = 0$ and $dp[0][a > 0] = +\infty$ to mark unreachable amounts. For denomination c_i, either use none, yielding $dp[i-1][a]$, or use at least one, yielding $1 + dp[i][a - c_i]$. The second expression deliberately reads the *current denomination row*: the remaining amount may still use that denomination.

$$\boxed{dp[i][a] = \min\{dp[i-1][a],\ 1 + dp[i][a - c_i]\}.}$$

The Take branch is available only when $a \geq c_i$.

```python
def coin_change_2d(coins, amount):
    INF = float('inf')
    m = len(coins)
    dp = [[INF] * (amount + 1) for _ in range(m + 1)]
    dp[0][0] = 0
    for i in range(1, m + 1):
        coin = coins[i - 1]
        for a in range(amount + 1):
            dp[i][a] = dp[i - 1][a]
            if a >= coin:
```

```
                dp[i][a] = min(dp[i][a], 1 + dp[i][a - coin])
    return -1 if dp[m][amount] == INF else dp[m][amount]
```

Why is ascending amount order useful even in the two-dimensional unbounded table? The Take branch reads $dp[i][a - c_i]$ on the *same row*, so smaller amounts must be computed first. In contrast, the 0/1 two-dimensional table can visit capacities in either direction because both branches read the previous row. When coin types are compressed into one array, ascending amount order naturally retains repeated use of the current coin.

Problem: Each denomination in `coins` may be used an unlimited number of times. Find the fewest coins summing to `amount` exactly; return -1 if impossible. Example: $[1, 2, 5]$ and amount 11 yield 3, from $5 + 5 + 1$.

$$dp[0] = 0, \quad dp[c > 0] = +\infty, \quad dp[c] = \min_{w \le c}\{dp[c - w] + 1\}.$$

```
def coin_change(coins, amount):
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0
    for coin in coins:
        for c in range(coin, amount + 1):
            dp[c] = min(dp[c], dp[c - coin] + 1)
    return -1 if dp[amount] == float('inf') else dp[amount]
```

Greedy counterexample: With denominations $[1, 3, 4]$ and amount 6, greedy chooses $4 + 1 + 1$ (three coins), while optimal is $3 + 3$ (two). **Complexity**: $O(mA)$ time, $O(A)$ space. **Perfect Squares (LC 279):** Set available denominations to $1, 4, 9, \ldots, \lfloor\sqrt{n}\rfloor^2$. The recurrence is identical and complexity is $O(n\sqrt{n})$.

Coin Change: Minimum Coins and Combination Counting Are Different Objectives

For coins $[1, 3, 4]$ and target amount 6, greedy takes 4 then two 1s, a total of three coins. DP also considers 3 as the final coin and discovers that $3 + 3$ needs only two. Let $d[x]$ be the fewest coins for amount x, with $d[0] = 0$ and unreachable states at $+\infty$. Enumerating the last coin for each amount gives $d[6] = \min(d[5] + 1, d[3] + 1, d[2] + 1) = 2$. If the question instead asks for all *unordered combinations*, you must no longer take the minimum of candidate values; count valid constructions and avoid counting different permutations of $3 + 1 + 1 + 1$ multiple times.

4.6 Canonical Problem 4: Combination Counts (LC 518) versus Ordered Sequences (LC 377)

Full problem statements: Combination counts versus ordered sequences

Problem A, Coin Change II. Given positive coin denominations and target A, each denomination may be used any number of times. Count distinct *unordered multisets of coins* with exact total A: changing the order of the same coins does not create another solution. For $[1, 2]$ and $A = 3$, the two solutions are $1 + 1 + 1$ and $1 + 2$.

Problem B, Combination Sum IV. Given positive integers that may be reused, count *ordered sequences* whose sum is exactly A. On the same input, $2+1$ is now different from $1+2$, so the count is three. Positive denominations ensure a finite maximum sequence length and rule out zero-sum cycles of arbitrary length.

Derive unordered counting from a two-dimensional table

Define $ways[i][a]$ as the number of combinations forming a using only the *first i denominations*. Start with $ways[0][0] = 1$ and other empty-denomination amounts equal to zero. Combinations using no current coin come from the previous row, $ways[i-1][a]$. Those using at least one current coin come from $ways[i][a-c_i]$ on the same row, because a combination already allowed to use denomination i may append another coin of that same type without creating a new ordering:

$$\boxed{ways[i][a] = ways[i-1][a] + \mathbf{1}_{a \geq c_i} ways[i][a-c_i].}$$

```python
def coin_combinations_2d(coins, amount):
    m = len(coins)
    dp = [[0] * (amount + 1) for _ in range(m + 1)]
    dp[0][0] = 1
    for i in range(1, m + 1):
        coin = coins[i - 1]
        for a in range(amount + 1):
            dp[i][a] = dp[i - 1][a]
            if a >= coin:
                dp[i][a] += dp[i][a - coin]
    return dp[m][amount]
```

Ordered sequences require a different subproblem. Define $count[a]$ to be the number of ordered sequences summing to a. Classify by the *last number* c: $count[a] = \sum_{c \leq a} count[a-c]$. At total 3, a sequence of sum 2 followed by 1 produces $(2,1)$, while a sequence of sum 1 followed by 2 produces $(1,2)$. Thus swapping the loop order is not a mysterious trick: it reflects whether the state keeps track of which denominations are already permitted. In either conventional counting problem, amount zero has exactly one empty selection, so initialize its count to one.

Problem A: Coin denominations can be reused without limit. Count combinations that make the target, ignoring order. With $[1,2]$ making 3, there are 2 combinations: $1+1+1$ and $1+2$. **State**: $dp[c]$ counts combinations, with $dp[0] = 1$ and *items outside, capacities increasing*:

```python
def coin_combinations(coins, amount):
    dp = [0] * (amount + 1)
    dp[0] = 1
    for coin in coins:
        for c in range(coin, amount + 1):
            dp[c] += dp[c - coin]
    return dp[amount]
```

Problem B: For the same input, count $1+2$ and $2+1$ as distinct sequences, giving 3 results. Put *amount in the outer loop* and enumerate the final number:

```python
def ordered_sequences(nums, target):
    dp = [0] * (target + 1)
    dp[0] = 1
    for c in range(1, target + 1):
        for x in nums:
            if x <= c:
                dp[c] += dp[c - x]
    return dp[target]
```

Complexity: Both take $O(mA)$ time and $O(A)$ space. **Difference:** Item-outer iteration incorporates denominations one by one and avoids recounting permutations of the same multiset; amount-outer iteration enumerates the last step and counts distinct orders separately. If the task allows negative numbers, zeros, or unlimited-length solutions, the number of sequences can be infinite, so the model needs additional restrictions.

4.7 Canonical Problem 5: Target Sum (LeetCode 494)

Full problem statement: Every array position must receive a sign

Given an array of nonnegative integers `nums` and an integer `target`, assign *either* + *or* − *to every indexed element*; no element may be omitted. Count the assignments whose signed sum is exactly the target. Equal numbers at different positions still have independent sign choices. With five ones and target 3, exactly four signs must be positive and one negative; the negative position can be any of the five, so the answer is 5.

Derive the algebraic reduction and a two-dimensional counting table

Interpret the positive-sign positions as members of a selected subset, and negative-sign positions as its complement. Let total unsigned sum be S, positive sum be P, and magnitude of negative sum be N. We need $P - N = T$ and $P + N = S$, hence $P = (S + T)/2$. The problem becomes: *how many 0/1 subsets of indexed elements have sum exactly* $G = (S + T)/2$*?*

Define $ways[i][a]$ as the number of choices using the first i array positions with sum exactly a. The initial value is $ways[0][0] = 1$, and the recurrence is

$$ways[i][a] = ways[i-1][a] + \mathbf{1}_{a \geq x_i}\, ways[i-1][a - x_i].$$

```python
def target_sum_2d(nums, target):
    S = sum(nums)
    if abs(target) > S or (S + target) % 2:
        return 0
    G = (S + target) // 2
    n = len(nums)
    dp = [[0] * (G + 1) for _ in range(n + 1)]
    dp[0][0] = 1
    for i in range(1, n + 1):
        x = nums[i - 1]
        for a in range(G + 1):
```

```
            dp[i][a] = dp[i - 1][a]
            if a >= x:
                dp[i][a] += dp[i - 1][a - x]
    return dp[n][G]
```

Unlike a maximization knapsack, the two branches are *added* because we count distinct selections. In the two-dimensional table, amount order is immaterial because both branches read the previous row; only a one-dimensional in-place version must scan downward. Be sure to test `nums=[0,0,1],target=1`: each zero can independently receive either sign, so four assignments are valid. Both taking and not taking a zero contribute one count, correctly doubling the result for each zero.

Problem: Assign a + or − sign to every nonnegative integer a_i to reach total T. Count the sign assignments. For $[1,1,1,1,1]$ and $T = 3$, the answer is 5. Let P be the sum of positively signed elements and N the sum of negatively signed elements. We have $P - N = T$ and $P + N = S$, hence

$$\boxed{P = (S + T)/2.}$$

If $|T| > S$ or $S + T$ is odd, no solutions exist. Otherwise this becomes **0/1 knapsack counting**; each element is selected at most once, but equal values at different indices are distinct choices:

```
def target_sum_ways(nums, target):
    S = sum(nums)
    if abs(target) > S or (S + target) % 2:
        return 0
    goal = (S + target) // 2
    dp = [0] * (goal + 1)
    dp[0] = 1
    for x in nums:
        for c in range(goal, x - 1, -1):
            dp[c] += dp[c - x]
    return dp[goal]
```

Zero elements: When $x = 0$, each sign assignment splits into positive and negative choices at that position, doubling the count; this is correct. **Complexity**: $O(n \cdot goal)$ time and $O(goal)$ space.

4.8 Advanced Models: Bounded, Group, and Multidimensional Knapsack

Complete problem statements for three advanced variants

Bounded knapsack. There are several item types, each with unit weight w_i, unit value v_i, and at most k_i available copies. With capacity W, maximize value. For $w = [2,4]$, $v = [3,5]$, $k = [3,2]$, and $W = 8$, taking two weight-2 units and one weight-4 unit gives value 11; two weight-4 units give 10, and three weight-2 units give 9. The inventory cap must be enforced; this is not an unbounded item.

Group knapsack. Items are partitioned into mutually exclusive groups, as when an investment strategy offers several versions and at most one version may be imple-

mented. For each group, choose none or at most one item. Two items from the same group cannot both be taken even when capacity would allow them.

Multidimensional knapsack (LC 474). Each binary string consumes a certain number of zeros and ones, with separate budgets M and N. Select as many strings as possible, each at most once. String `10` consumes $(1, 1)$; `0001` consumes $(3, 1)$. Keeping only string length cannot establish feasibility with these two independent budgets.

Uncompressed states and dependency directions

For bounded knapsack, use $dp[i][c]$: after the first i item types, enumerate $k = 0, 1, \ldots, k_i$ copies of the current type, reading $dp[i-1][c - kw_i]$ from the *previous row* for every candidate. Binary decomposition into 0/1 batches is a computational optimization, not a different definition of the problem.

For group knapsack, $dp[i][c]$ means the first i *groups*, not the first i individual items. The Take and Skip choices for the current group all read previous-group states, preventing two items in the same group from being taken.

For two-resource knapsack, the uncompressed state is $dp[i][a][b]$: the best answer after processing the first i strings with zero budget a and one budget b. If string i contains z_i zeros and o_i ones,

$$dp[i][a][b] = \max\{dp[i-1][a][b],\ 1 + dp[i-1][a - z_i][b - o_i]\},$$

where the Take branch requires $a \geq z_i$ and $b \geq o_i$. In the three-dimensional table, either capacity may be traversed in either direction because the previous i-layer is read. When the item layer is removed and a single two-dimensional $dp[a][b]$ is updated *in place*, **both resource dimensions must be traversed downward** so that no string can be reused. Here "two-dimensional knapsack" means two resource limits; it is not the same as the standard $dp[i][c]$ table with one item dimension and one capacity dimension.

Bounded: Item type i may be selected at most k_i times:

$$dp_i[c] = \max_{0 \leq j \leq \min(k_i, \lfloor c/w_i \rfloor)} \{dp_{i-1}[c - jw_i] + jv_i\}.$$

Direct triple loops may be expensive. Binary grouping into $1, 2, 4, \ldots,$ remainder converts each inventory into $O(\log(k_i + 1))$ 0/1 groups.

```python
def bounded_knapsack(weights, values, counts, W):
    dp = [0] * (W + 1)
    for w, v, k in zip(weights, values, counts):
        block = 1
        while k:
            take = min(block, k)
            ww, vv = w * take, v * take
            for c in range(W, ww - 1, -1):
                dp[c] = max(dp[c], dp[c - ww] + vv)
            k -= take
            block *= 2
    return dp[W]
```

Group: At most one item per group. Use a snapshot of the preceding group's state for each group; otherwise two items from the same group may accidentally be selected.

```python
def group_knapsack(groups, W):
    dp = [0] * (W + 1)
    for group in groups:  # group: [(weight,value), ...]
        old = dp[:]
        for c in range(W + 1):
            dp[c] = old[c]
            for w, v in group:
                if w <= c:
                    dp[c] = max(dp[c], old[c - w] + v)
    return dp[W]
```

2D Knapsack (LC 474): Every binary string consumes a certain number of zeros and ones under budgets M, N, respectively. Maximize the number of selected strings.

```python
def ones_zeroes(strs, M, N):
    dp = [[0] * (N + 1) for _ in range(M + 1)]
    for s in strs:
        z, o = s.count('0'), s.count('1')
        for a in range(M, z - 1, -1):
            for b in range(N, o - 1, -1):
                dp[a][b] = max(dp[a][b], dp[a-z][b-o] + 1)
    return dp[M][N]
```

Complexity: $O(\sum |s| + \#strs \cdot MN)$ time and $O(MN)$ space.

Common Pitfall

Remember two independent rules: (1) With items as the outer loop and in-place one-dimensional DP, 0/1 capacity scans downward while unbounded scans upward. (2) In *counting* with unlimited items, item-outer iteration counts unordered combinations, whereas amount-outer iteration counts ordered sequences. Do not collapse "ascending/descending" and "outer/inner loop" into a single rule.

5 Family Four: Two-Sequence Alignment and Grid Paths

5.1 Recognition: How Do Two Indices or Coordinates Advance?

Clues: Matching two strings or arrays, editing A into B, moving from the top-left of a matrix to the bottom-right, moving only right/down, or avoiding a forbidden boundary. A common state is $dp[i][j]$, describing the first i elements of sequence A together with the first j elements of B, or the result of reaching grid cell (i, j).

Two-sequence technique: Focus on the final characters: consume both tails (diagonal), consume only A's tail (above), or consume only B's tail (left). **Grid technique:** Identify the legal predecessors of each cell. When paths cannot move backward, the dependencies form a DAG that can be filled in topological order.

5.2 Recognition in Detail: Why Do Two Indices Often Mean Two-Dimensional DP?

This family contains two branches: **comparing two sequences** and **moving on a two-dimensional grid**. Both use $dp[i][j]$, but the indices have different meanings. Do not blindly reuse a recurrence across them.

1. **"Turn string A into string B with the fewest changes":** You must know how many characters of A and B have been processed; A's index alone is insufficient. A classic example transforms `horse` into `ros` by insertions, deletions, and replacements, with answer 3. This is Edit Distance.

2. **"How long is the common pattern in two data sequences?":** Relative order must be maintained, but matched elements need not be contiguous. For example, the LCS of `abcde` and `ace` is `ace`. The state uses prefix lengths (i, j) and combines candidates with max.

3. **"Find a path from top-left to bottom-right":** You may move right or down, and are asked for path count, lowest cost, or highest payoff. Here (i, j) is a *grid coordinate*; predecessors are up and left. Use addition for counting and min for shortest paths.

4. **"A walk must not cross the diagonal":** For example, move right or up from $(0, 0)$ to (n, n) without ever taking more upward than rightward steps. This adds a *feasibility constraint* to ordinary grid paths. Invalid states contribute 0; use DP or the Catalan formula.

5. **"Pattern matching with wildcards":** The regex symbols `.` and $*$ are not literal characters. Decide *how many input characters a pattern token matches*. The state still consists of two prefix lengths, but its transition is no longer simply a choice of three neighboring cells.

5.2.1 Two Sequences: How Do the Final Characters Partition the Problem?

When defining $dp[i][j]$, specify that it processes `a[:i]` and `b[:j]`; therefore their last characters are `a[i-1]` and `b[j-1]`. Any final matching or editing step must account for these two tails. Typical choices are:

- Consume both tails and go to $(i-1, j-1)$: match or replace.
- Consume only A's tail and go to $(i-1, j)$: delete a character from A, depending on the problem's semantics.
- Consume only B's tail and go to $(i, j-1)$: insert a character into A or ignore a tail of B.

These three directions are not a mnemonic pulled from nowhere. They exhaust *what the final operation does to the two sequences*. In Edit Distance, equal tails match at no cost; unequal tails correspond to replacement, insertion, or deletion, each adding cost 1 along the corresponding direction.

5.2.2 Grid Paths: Entering versus Leaving a Cell

Consider (i, j). If it can only be entered from above $(i-1, j)$ or from the left $(i, j-1)$, the *number of paths* reaching it is the sum of the two predecessor counts. With costs assigned to grid cells, the *minimum cost to reach the cell* becomes

$$dp[i][j] = cost[i][j] + \min\{dp[i-1][j], dp[i][j-1]\}.$$

If unrestricted left, right, up, and down movements are allowed, dependencies can contain cycles. A single forward grid DP may then be inappropriate; BFS, Dijkstra, or another graph algorithm is usually more suitable. **A key requirement for ordinary grid DP is that dependencies can be evaluated in an acyclic order.**

5.2.3 Which Row Does a Rolling One-Dimensional Array Actually Store?

When processing rows top-to-bottom and columns left-to-right, before updating column j of a new row, `dp[j]` holds the *previous row's value in the same column*, while `dp[j-1]` already holds the *current row's left neighbor*. Thus `dp[j] += dp[j-1]` implements the original two-dimensional recurrence exactly. For LCS, if the transition also needs the upper-left diagonal $(i-1, j-1)$, save that old diagonal value (or keep two complete rows).

Common Pitfall

Do not confuse **subsequence (not necessarily contiguous)** with **substring (contiguous)**. Nor should both indices of `dp[i][j]` always be interpreted as "time and capital." They could mean processed prefix lengths of two strings, grid coordinates, or two players' resource positions. State names are mere symbols; the definition supplies their meaning.

5.3 Canonical Problem 1: Unique Paths (LeetCode 62)

Full problem statement: Why are there only two predecessors?

A robot starts in the upper-left cell $(0,0)$ of an $m \times n$ grid. At each step it may move *exactly one cell right or one cell down*; it may never move back, move diagonally, or leave the grid. It stops at the bottom-right cell $(m-1, n-1)$. Count distinct movement sequences. For $m = 3, n = 7$, every path contains precisely two downward and six rightward moves, and there are 28 distinct orders. If the grid has only one row or column, there is exactly one path. In a variation with blocked cells, any route through an obstacle is invalid.

Hand-fill a tiny grid from boundary to interior

In a 3×3 obstacle-free grid, each cell in the first row has one route (always move right), and each cell in the first column likewise has one route (always move down). The center cell $(1,1)$ has $1 + 1 = 2$ routes, from above and from the left. Cells $(1,2)$ and $(2,1)$ each have three routes, and the destination $(2,2)$ has $3 + 3 = 6$. Thus the completed table is

$$\begin{pmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \\ 1 & 3 & 6 \end{pmatrix}.$$

We *add disjoint predecessor counts* rather than take their maximum. If the objective changes to minimal path cost, the transition instead takes a `min` over predecessors and adds the current cell's cost. The full two-dimensional table must calculate both upper and left predecessor values before reading them. In a single rolling row, `dp[j]` still means the previous row's current column, whereas `dp[j-1]` has already become the current row's left neighbor.

Problem: In an $m \times n$ grid, start at $(0,0)$ and move only right or down. How many paths reach the bottom-right corner? In a 3×7 grid, the answer is 28. **State**: $dp[i][j]$ counts paths from the start to (i, j). The final move can only come from $(i-1, j)$ or $(i, j-1)$, hence

$$\boxed{dp[i][j] = dp[i-1][j] + dp[i][j-1]}.$$

Base: Every cell in the first row and first column has exactly one path when there are no obstacles.

```python
def unique_paths(m, n):
    dp = [[1] * n for _ in range(m)]
    for i in range(1, m):
        for j in range(1, n):
            dp[i][j] = dp[i - 1][j] + dp[i][j - 1]
    return dp[m - 1][n - 1]
```

Complexity: $O(mn)$ time, $O(mn)$ space. **Rolling optimization:**

```python
def unique_paths_rolling(m, n):
    dp = [1] * n
    for _ in range(1, m):
        for j in range(1, n):
            dp[j] += dp[j - 1]
    return dp[-1]
```

Space becomes $O(n)$. **Mathematical improvement:** Each path consists of $m-1$ downward moves and $n-1$ rightward moves. Choose the positions of the downward moves:

$$\boxed{\binom{m+n-2}{m-1}}.$$

Use `math.comb` to avoid building a DP table. **Obstacles:** Set $dp[i][j] = 0$ if the cell is blocked; otherwise add the values from above and left, handling the first row and column carefully.

```python
def paths_with_obstacles(grid):
    m, n = len(grid), len(grid[0])
    dp = [0] * n
    dp[0] = 0 if grid[0][0] else 1
    for i in range(m):
        for j in range(n):
            if grid[i][j]:
                dp[j] = 0
            elif j > 0:
                dp[j] += dp[j - 1]
    return dp[-1]
```

5.4 Canonical Problem 2: Longest Common Subsequence (LCS, LeetCode 1143)

Full problem statement: Subsequence is not substring

Given strings A and B, find the length of their *longest common subsequence* (LCS). You may delete any characters from either string, but the remaining characters must preserve their original relative order; they do *not* need to be adjacent. For $A =$ `abcde` and $B =$ `ace`, the common subsequence `ace` has length 3. A *common substring*, by contrast, must consist of consecutive characters and needs a different state transition. Either input may be empty, giving answer zero.

Derive the matching and mismatching cases by hand

Let $dp[i][j]$ be the LCS length for the first i characters of A and the first j of B. If the final characters agree, an optimal common subsequence can use that matching pair, giving $dp[i-1][j-1]+1$. If they disagree, they cannot both be used as the last symbol of the same common subsequence, so at least one must be discarded: take $\max(dp[i-1][j], dp[i][j-1])$. For $A =$ `abc` and $B =$ `ac`, $dp[1][1] = 1$ from matching a's, $dp[2][1] = 1$ when b fails to match a, and $dp[3][2] = dp[2][1] + 1 = 2$ when the final c's match. The answer is subsequence `ac` of length 2.

The full table is filled from top-left toward bottom-right so the top, left, and diagonal predecessor cells are available. When rolling into one row, the *old upper-left diagonal* must be stored separately (for example in `diag`), because it would otherwise be overwritten during the current row.

Problem: Given two strings, find the length of their longest common subse-

quence, which preserves relative order without requiring adjacent characters. For `abcde` and `ace`, one longest common subsequence is `ace`, of length 3.

State: $dp[i][j]$ is the LCS length of the first i characters of A and first j of B. **Base**: $dp[0][*] = dp[*][0] = 0$. If the last characters match, they can be paired; otherwise both last characters cannot simultaneously be used as the final matching pair:

$$dp[i][j] = \begin{cases} dp[i-1][j-1] + 1, & A_{i-1} = B_{j-1}, \\ \max\{dp[i-1][j], dp[i][j-1]\}, & A_{i-1} \neq B_{j-1}. \end{cases}$$

```python
def lcs(a, b):
    m, n = len(a), len(b)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if a[i-1] == b[j-1]:
                dp[i][j] = 1 + dp[i-1][j-1]
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    return dp[m][n]
```

Complexity: $O(mn)$ time and $O(mn)$ space. For length only, roll the table to $O(\min(m, n))$ space by retaining a previous row and saving the old diagonal value for each cell. To reconstruct the actual LCS, keep predecessor pointers or the full table, or use Hirschberg's divide-and-conquer algorithm to reconstruct with $O(mn)$ time and $O(\min(m, n))$ auxiliary space.

```python
def lcs_rolling(a, b):
    if len(b) > len(a):
        a, b = b, a
    dp = [0] * (len(b) + 1)
    for x in a:
        diag = 0
        for j, y in enumerate(b, 1):
            up = dp[j]
            if x == y:
                dp[j] = diag + 1
            else:
                dp[j] = max(dp[j], dp[j - 1])
            diag = up
    return dp[-1]
```

LCS: Calculating a Two-Dimensional Cell Step by Step

Take $A =$ `abcde` and $B =$ `ace`. First compare prefix `a` with `a`: the tails match, so $dp[1][1] = dp[0][0] + 1 = 1$. Next compare `ab` with `a`: the tails b and a do not match, so either omit A's last character from the shared subsequence or omit B's last character. Thus $dp[2][1] = \max(dp[1][1], dp[2][0]) = 1$. Continue until $dp[5][3] = 3$, corresponding to `ace`. This demonstrates that the indices i, j represent **the prefix lengths already processed**, not the zero-based indices of the elements themselves. This difference of one is a frequent source of off-by-one bugs.

5.5 Canonical Problem 3: Edit Distance (LeetCode 72)

Full problem statement: Each edit changes one character

Given strings A and B, find the minimum number of unit-cost *single-character insertions, deletions, or replacements* needed to transform the entire A into exactly B. Operations may occur at any position. For $A =$ `horse` and $B =$ `ros`, replace h by r (`rorse`), delete the extra r (`rose`), and delete the final e (`ros`): three edits. Transforming an empty string into B requires $|B|$ insertions.

How the last edit produces the three-direction recurrence

At state $dp[i][j]$, transform the first i characters of A into the first j of B. If their final characters match, preserve that pair at no cost and move diagonally to $dp[i-1][j-1]$. Otherwise three possibilities exhaust the last operation. **Replace** the final character of A by that of B: $dp[i-1][j-1]+1$. **Delete** the final character of A: $dp[i-1][j]+1$. **Insert** the final character of B: first form the shorter target prefix, $dp[i][j-1]+1$. For $A =$ `ab`, $B =$ `ac`, the first a is already matched and replacing b with c gives $dp[2][2] = 1$.

Remember that $dp[i][0] = i$ and $dp[0][j] = j$: these boundaries differ from LCS, where an empty string has common subsequence length zero. To output an actual edit script rather than just its length, trace the optimal transitions backward from the bottom-right corner of the completed table.

Problem: What is the fewest number of single-character *insertions, deletions, and replacements* needed to turn string A into B? For `horse`→`ros`, the answer is 3.

State: $dp[i][j]$ is the fewest operations needed to convert the first i characters of A into the first j of B. Empty-string boundaries are $dp[i][0] = i$ and $dp[0][j] = j$. Equal tails cost nothing; otherwise work backward from three possible *final operations*: replace $(i-1, j-1)$, delete $(i-1, j)$, or insert $(i, j-1)$.

$$dp[i][j] = \begin{cases} dp[i-1][j-1], & A_{i-1} = B_{j-1}, \\ 1 + \min\{dp[i-1][j-1], dp[i-1][j], dp[i][j-1]\}, & \text{otherwise.} \end{cases}$$

```python
def edit_distance(a, b):
    m, n = len(a), len(b)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(m + 1):
        dp[i][0] = i
    for j in range(n + 1):
        dp[0][j] = j
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if a[i-1] == b[j-1]:
                dp[i][j] = dp[i-1][j-1]
            else:
                dp[i][j] = 1 + min(dp[i-1][j-1],
                                   dp[i-1][j], dp[i][j-1])
    return dp[m][n]
```

Complexity: $O(mn)$ time and $O(mn)$ space; if only distance is needed, use two rows and $O(\min(m, n))$ space. **Follow-up:** With unequal edit costs, replace +1 by the appro-

priate insertion, deletion, or replacement cost; this yields weighted sequence alignment.

Edit Distance: How Do Replace, Insert, and Delete Map to Predecessors?

Convert `horse` into `ros`. One optimal edit sequence is: replace h with r, producing `rorse`; delete the third character r, producing `rose`; delete the final e, producing `ros`. Total: three operations. Define $dp[i][j]$ consistently as "minimum cost to turn the first i characters of A into the first j characters of B." Equal last characters use $dp[i-1][j-1]$ directly; otherwise replacement adds 1 to the diagonal, deletion adds 1 to the cell above, and insertion adds 1 to the cell on the left. This is a *classification by the final edit operation*, not an arbitrary three-direction formula.

5.6 Canonical Problem 4: Regular Expression Matching (LeetCode 10)

Full problem statement: The star repeats the preceding pattern atom

Given text `s` and pattern `p`, decide whether the *entire text* matches the entire pattern, not merely a substring. Patterns contain ordinary characters, a dot `.` matching *exactly one* arbitrary character, and a star `*` repeating the *immediately preceding* pattern atom *zero or more times*. For `s="aab"`, `p="c*a*b"`, the result is True: `c*` matches zero c's, `a*` matches two a's, and the final b matches b. Importantly, `*` does not mean "match any text" by itself.

Why the star transition does not simply consume one character

Let $dp[i][j]$ mean that the first i text characters match the first j pattern characters. For an ordinary final atom, it must match the text's final character and then reduce to $dp[i-1][j-1]$. When the pattern ends in `x*`, there are two fundamentally different cases: **zero x's**, discarding the entire trailing `x*` pair and reading $dp[i][j-2]$; or **at least one x**, consuming the final text character (if it matches x, or x is a dot) while *retaining* the pattern `x*`, reading $dp[i-1][j]$ so it can match more x's later.

The empty string `""` matches `"a*b*"` because each starred atom may occur zero times; it does not match `"ab*"` because the unstarred a cannot disappear. Hence the first row of the DP table needs valid zero-occurrence transitions; do not permanently initialize every cell except $dp[0][0]$ to False. Complexity is $O(|s||p|)$. A different regex language with `?`, character classes, or arbitrary wildcard semantics would require checking its exact rules before reusing these transitions.

Problem: Pattern p consists of ordinary characters, `.` (any single character), and `*` (zero or more repetitions of the preceding element). Determine whether text s matches the *entire* pattern. For example, `s="aab", p="c*a*b"` is True.

State: $dp[i][j]$ indicates whether the first i text characters match the first j pattern characters completely. If the pattern's last character is not `*`, its tail must match the text tail, and $dp[i-1][j-1]$ must hold. If it is `*`, either repeat the preceding element zero times (remove the last two pattern tokens) or at least once (consume a text

character while retaining the pattern position):

$$dp[i][j] = dp[i][j-2] \\ \vee \left(i > 0 \wedge \text{match}(s_{i-1}, p_{j-2}) \wedge dp[i-1][j]\right),$$

The expression above applies when $p_{j-1} = *$; otherwise $dp[i][j] = \text{match}(s_{i-1}, p_{j-1}) \wedge dp[i-1][j-1]$.

```python
def regex_match(s, p):
    m, n = len(s), len(p)
    dp = [[False] * (n + 1) for _ in range(m + 1)]
    dp[0][0] = True
    for i in range(m + 1):
        for j in range(1, n + 1):
            if p[j-1] == '*':
                if j >= 2:
                    dp[i][j] = dp[i][j-2]
                    if i and (p[j-2] == '.' or p[j-2] == s[i-1]):
                        dp[i][j] |= dp[i-1][j]
            elif i and (p[j-1] == '.' or p[j-1] == s[i-1]):
                dp[i][j] = dp[i-1][j-1]
    return dp[m][n]
```

Assumption: The pattern is valid (* never starts the pattern and modifies a preceding valid element). **Complexity**: $O(mn)$ time and space. **Key insight:** A trailing * can either match zero copies or consume one character while remaining available to match more.

5.7 Canonical Problem 5: Catalan Numbers and Dyck Paths (Diagonal Constraint)

Full problem statement: Counting grid paths under a positional constraint

Start at lattice point $(0, 0)$ and move to (n, n) using only right moves $(x + 1, y)$ and up moves $(x, y+1)$. After every move, require $y \leq x$: the route may touch the diagonal but must never pass *above* it. Count legal routes. For $n = 3$ there are five. Without the constraint there would be $\binom{6}{3} = 20$ paths, many of which cross the forbidden region.

A small enumeration and its connection to parentheses

For $n = 2$, take exactly two right and two up moves, with the number of up moves in every prefix no greater than the number of right moves. Only `RRUU` and `RURU` are legal, so $C_2 = 2$. Write right as `(` and up as `)`: the inequality $y \leq x$ becomes "no prefix has more closing than opening brackets," and the endpoint demands equal numbers, exactly the rule for a balanced-parentheses string. This explains the Dyck-path/Catalan correspondence.

A two-dimensional grid DP remains useful when extra forbidden points or edge weights are introduced. Make $dp[x][y] = 0$ whenever $y > x$; otherwise add the allowed predecessor counts. For the unconstrained variation-free form, $C_n = \binom{2n}{n}/(n+1)$ gives a closed-form count. The closed form exploits structure, whereas the DP remains

flexible under further constraints—a useful distinction for random-walk interview variants.

Problem: Move from $(0,0)$ to (n,n) by right and upward unit steps, requiring $y \le x$ at every point. Count valid paths. For $n = 3$, the answer is 5.

State: $dp[x][y]$ counts paths reaching the point without ever crossing the boundary. If $y > x$, set $dp[x][y] = 0$; otherwise $dp[x][y] = dp[x-1][y] + dp[x][y-1]$, treating out-of-bounds terms as zero. Initialize $dp[0][0] = 1$.

```
def dyck_paths(n):
    dp = [[0] * (n + 1) for _ in range(n + 1)]
    dp[0][0] = 1
    for x in range(n + 1):
        for y in range(min(x, n) + 1):
            if x == y == 0:
                continue
            dp[x][y] = (dp[x-1][y] if x else 0) + \
                       (dp[x][y-1] if y else 0)
    return dp[n][n]
```

Complexity: $O(n^2)$ time and space. **Mathematical upgrade:** The reflection principle yields the nth Catalan number:

$$\boxed{C_n = \frac{1}{n+1}\binom{2n}{n}}.$$

This connects valid parenthesis strings, shapes of binary trees, and constrained random walks. **Caveat:** With only this diagonal restriction there is a closed form; obstacles or complex edge costs are what make grid DP genuinely necessary.

5.8 Supplementary Canonical Problem: Word Break (LeetCode 139)

Full problem statement and examples

Given a string `s` and a dictionary `wordDict` of nonempty words, determine whether the *entire string* can be split into one or more consecutive segments, each contained in the dictionary. A dictionary word can be used repeatedly. For `s="leetcode", wordDict=["leet","code"]`, the answer is True. For `"applepenapple"` with words `["apple","pen"]`, it is True because `apple` can repeat. For `"catsandog"` with `["cats","dog","sand","and","cat"]`, the answer is False.

Reason backward from the final word to a feasible prefix

Define $dp[i]$ to say whether the first i characters `s[:i]` can be segmented completely. For each endpoint i, enumerate the start j of the final word. If $dp[j]$ is True and `s[j:i]` is a dictionary word, append that word to the valid prefix and set $dp[i]$ = True. The base case $dp[0]$ = True is essential: otherwise the first word could never start. For `leetcode`, $dp[4]$ becomes True on `leet`; $dp[8]$ becomes True because $dp[4]$ is True and `code` is valid; intermediate prefixes remain False.

The two nested split loops generate $O(n^2)$ candidate pairs, but Python substring slicing may cost $O(n)$ per candidate, making a naive implementation $O(n^3)$ in the worst case. Restrict potential word lengths, group dictionary words by length, or use a Trie to improve performance. A variation asking for the *fewest words* replaces Boolean feasibility with a minimum-count state; a variation asking for the *number of segmentations* replaces logical OR with addition of counts.

Given a string and a dictionary, determine whether it can be split into dictionary words, each of which may be reused. Let $dp[i]$ indicate whether the first i characters can be segmented:

$$dp[0] = True, \quad dp[i] = \bigvee_{0 \le j < i} \{dp[j] \wedge (s[j:i] \in D)\}.$$

```python
def word_break(s, dictionary):
    words = set(dictionary)
    n = len(s)
    dp = [False] * (n + 1)
    dp[0] = True
    for i in range(1, n + 1):
        for j in range(i):
            if dp[j] and s[j:i] in words:
                dp[i] = True
                break
    return dp[n]
```

Complexity: There are $O(n^2)$ candidate intervals, but Python slices take $O(i-j)$ time, giving a naive worst-case bound of $O(n^3)$. Restricting candidate word lengths or using a trie can reduce practical cost. This is **prefix-partition DP**, training the question "where does the last valid segment begin?" much like two-sequence DP.

6 Family Five: Interval Merging and Zero-Sum Games

6.1 Recognition Clues and Common Structure

Clues: You may take elements only from either end; each move merges adjacent intervals; deleting an element changes which others are adjacent; two optimal players alternate moves. State $dp[l][r]$ describes a closed or open interval, typically solved from shorter intervals to longer ones.

Two common patterns: (A) *Remove from either end*: choose left or right, leaving a smaller interval for the opponent. (B) *Merge or partition an interval*: enumerate the final split point k, or the last element k removed, so the left and right portions become independent.

6.2 Recognition in Detail: Is the Remainder One Interval or an Arbitrary Set?

This family is distinctive because **an operation usually affects a contiguous interval $[l,r]$, leaving a smaller interval or splitting it into two independent intervals**. When both players are described as intelligent, we must additionally anticipate the opponent's optimal response.

1. **Remove a number from either end:** "There is a row of coins; each player takes only the leftmost or rightmost coin, alternating turns, and the higher score wins." Taking the left leaves $[l+1,r]$; taking the right leaves $[l,r-1]$. The adversarial nature explains why the recurrence *subtracts the opponent's optimal score difference*.

2. **Merge adjacent intervals:** "Each time two neighboring files are merged, you pay the sum of their lengths; minimize total cost." Before $[l,r]$ is finally merged, its left and right portions must have been merged separately for some cut k. Enumerating k gives the recurrence.

3. **Neighbors change after deletion:** "The points for bursting a balloon depend on its currently adjacent balloons." If you enumerate the *first* balloon burst, later neighbors are hard to pin down. Enumerating the *last* balloon burst inside an interval instead fixes the boundaries and makes the left and right subintervals independent.

4. **Longest palindromic subsequence:** "Delete some characters while preserving relative order to obtain the longest palindrome." Compare the endpoints: if equal, they can sit at the two ends of a palindrome; if unequal, at least one endpoint is not used in an optimal solution.

5. **Matrix chain multiplication:** "Change parenthesization of a sequence of compatible matrix products to minimize scalar multiplications." Independently op-

timize the left and right subchains, then enumerate their final multiplication boundary.

6.2.1 How Is Interval DP Different from Ordinary Prefix DP?

Removing a middle element from an array may leave a remainder that cannot be described by "only the first i elements." Interval DP typically keeps both the *left boundary l and right boundary r* to identify which elements remain. Conversely, selecting arbitrary nonadjacent elements without interval-dependent constraints may not require interval DP.

6.2.2 Net Score Difference in Zero-Sum Games: Why Is It Simpler Than Tracking Player Totals?

Suppose coins are $[1, 5, 2]$ and the current player may take either 1 or 2. Taking 1 leaves $[5, 2]$ to the opponent. The opponent can then achieve a net advantage of 3 (take 5 and leave 2), so the current player's total net advantage is $1 - 3 = -2$. Taking 2 leaves $[1, 5]$, where the opponent can gain a net advantage of 4, giving $2 - 4 = -2$. Thus the current player can do no better than -2 and the first player loses. This formulation avoids explicitly storing "player A's turn or player B's turn," since every $D(l, r)$ measures *the player about to move relative to their opponent*.

6.2.3 Why Does Interval DP Usually Iterate over Length First?

Suppose $dp[l][r]$ needs $dp[l+1][r]$ and $dp[l][r-1]$, both one element shorter than the current interval of length $r - l + 1$. Processing lengths 1, 2, 3, and so forth naturally ensures prerequisites are available. If the algorithm also enumerates a cut $k \in [l, r-1]$, there are $O(n^2)$ interval states and $O(n)$ candidate cuts per state, often giving $O(n^3)$ time. Complexity depends on both the number of states and candidate decisions; not every interval DP takes $O(n^3)$.

6.2.4 Choosing Boundaries Carefully: Closed versus Open Intervals

Taking coins from the ends naturally uses closed intervals $[l, r]$. Burst Balloons is more naturally modeled with *open intervals* (l, r), because l and r are surviving boundary balloons. A closed interval with one element may have that element's value, whereas an open interval with no internal balloon has value 0. **Boundary definitions determine transition indices; sketch a three- or four-element example before coding.**

6.3 Canonical Problem 1: Taking Coins from the Ends (Predict the Winner, LeetCode 486)

Full problem statement: Two optimal players, only two available ends

A row of n coins has integer values $a_0, \ldots, a_{n-1}$. Players alternate turns and must take *exactly one* coin from either end of the *remaining contiguous row*. Each taken coin

is added to that player's score. Continue until all coins are taken. Both players see the entire row and play optimally. Determine whether the first player can guarantee a final score *at least as large* as the second player's, or return the largest net scoring advantage attainable against an optimal opponent. For $[1, 5, 2]$ the first player loses; for $[1, 5, 233, 7]$ the first player can win.

Why net advantage removes the need for a turn variable

Let $D(l, r)$ be the maximum final score difference *current player minus opponent* when the remaining coins occupy $[l, r]$. If the current player takes left coin a_l, the opponent takes the next turn on $[l + 1, r]$ and enjoys optimal advantage $D(l + 1, r)$. The current player's resulting advantage is thus $a_l - D(l + 1, r)$; taking right is analogous. The negative sign reflects the zero-sum reversal of roles.

For $[1, 5, 2]$, single-coin interval values are the coin values. Length-two intervals give $D(0, 1) = \max(1 - 5, 5 - 1) = 4$ and $D(1, 2) = \max(5 - 2, 2 - 5) = 3$. Length three gives $D(0, 2) = \max(1 - 3, 2 - 4) = -2$, showing that the first player cannot guarantee a win. Fill intervals by lengths 1, 2, ..., n: every longer interval only depends on already computed shorter intervals. If skipping a turn, changing the scoring rule, or randomness in the opponent's behavior becomes possible, revise the actions and terminal values rather than blindly applying the same formula.

Problem: Two players optimally alternate taking the leftmost or rightmost coin from a row of nonnegative values $a_0, \ldots, a_{n-1}$. Does the first player finish with at least as many points as the second, or what is the greatest net advantage? For $[1, 5, 2]$ the first player loses (maximum net difference -2); for $[1, 5, 233, 7]$ the first player wins.

State: $D(l, r)$ is the **maximum number of points by which the player whose turn it is** can lead their opponent on the remaining interval $[l, r]$ (current player's score minus opponent's score). This avoids a separate turn dimension.

$$\boxed{D(l, r) = \max\{a_l - D(l + 1, r),\ a_r - D(l, r - 1)\}}, \quad D(i, i) = a_i.$$

Why subtract? If I take a_l, the opponent can gain advantage $D(l+1, r)$ in the remaining game, so my eventual net advantage is $a_l - D(l + 1, r)$.

Top-Down—recursion closely mirrors the game tree:

```
from functools import cache

def winner_topdown(nums):
    n = len(nums)
    @cache
    def diff(l, r):
        if l == r:
            return nums[l]
        return max(nums[l] - diff(l + 1, r),
                   nums[r] - diff(l, r - 1))
    return diff(0, n - 1) >= 0
```

Bottom-Up—in increasing order of interval length:

```
def max_net_score(coins):
    n = len(coins)
```

```
    if not n:
        return 0
    dp = [[0] * n for _ in range(n)]
    for i in range(n):
        dp[i][i] = coins[i]
    for length in range(2, n + 1):
        for l in range(n - length + 1):
            r = l + length - 1
            dp[l][r] = max(coins[l] - dp[l+1][r],
                           coins[r] - dp[l][r-1])
    return dp[0][n-1]
```

Complexity: $O(n^2)$ time and $O(n^2)$ space. **Space optimization:** Iterate l downward and r upward. One-dimensional `dp[r]` then contains $D(l+1, r)$ before its update, while `dp[r-1]` already contains $D(l, r-1)$, reducing space to $O(n)$:

```
def max_net_score_rolling(a):
    n = len(a)
    if not n:
        return 0
    dp = a[:]
    for l in range(n - 2, -1, -1):
        for r in range(l + 1, n):
            dp[r] = max(a[l] - dp[r], a[r] - dp[r-1])
    return dp[-1]
```

Advanced variation: If the opponent is not necessarily optimal, the same zero-sum minimax recurrence may not describe the problem. Negative coin values can still be handled by the net-difference definition, but specify whether all coins must be taken.

6.4 Canonical Problem 2: Burst Balloons (LeetCode 312)

Full problem statement: Neighbors change after each balloon disappears

You are given a row of balloons with positive integer values. On each move, burst any remaining balloon i and earn the product of its *current* left-neighbor value, its own value, and its *current* right-neighbor value. If there is no balloon on one side, use virtual boundary value 1. After a burst the balloon disappears, making its former neighbors adjacent. Burst *all* balloons while maximizing total score. For $[3, 1, 5, 8]$ the optimum is 167. Because adjacency changes with burst order, sorting fixed local products greedily is invalid.

Choose the last balloon to derive an open-interval state

Choosing the *first* balloon is inconvenient because its removal changes subsequent neighbors and destabilizes interval boundaries. Reverse the logic: retain two boundary balloons l, r and burst everything *strictly between them*. Suppose the last interior balloon to burst is k. Because l and r remain in place up to that instant, its final reward is exactly $b_l b_k b_r$. Before that, interiors (l, k) and (k, r) can be optimized independently, producing the open-interval recurrence.

For the tiny example $[3,1]$, augment the list to $[1,3,1,1]$. Bursting 3 first earns $1 \cdot 3 \cdot 1 = 3$, then bursting 1 earns 1, total 4. Bursting 1 first earns $1 \cdot 1 \cdot 3 = 3$, then bursting 3 earns 3, total 6. The optimum is 6. Being explicit that the interval is *open* prevents accidentally bursting or double-counting the boundary balloons inside a subproblem.

Problem: Balloons carry positive integers a_i. Bursting balloon i earns a_i times the values of its *currently adjacent balloons*; neighbors close the gap afterward. Treat missing outer neighbors as 1. Maximize total points from bursting all balloons. For $[3,1,5,8]$, the answer is 167.

Why not enumerate the first balloon burst? The first burst changes later adjacency, so the remaining subproblems are hard to separate. The key is to **reason backward and enumerate the last balloon burst in an interval**. When balloon k is burst last, boundaries l, r remain in place, and the final reward is fixed at $a_l a_k a_r$.

Pad the array at both ends with 1, writing $b = [1] + a + [1]$. Define $dp[l][r]$ as the maximum score from **bursting only balloons strictly inside the open interval** (l,r), leaving its endpoints intact. If $r = l + 1$, there are no internal balloons, so its value is 0. Enumerate the last internal balloon k:

$$\boxed{dp[l][r] = \max_{l<k<r}\{dp[l][k] + dp[k][r] + b_l b_k b_r\}.}$$

```python
def max_coins(nums):
    b = [1] + nums + [1]
    m = len(b)
    dp = [[0] * m for _ in range(m)]
    for gap in range(2, m):
        for l in range(m - gap):
            r = l + gap
            for k in range(l + 1, r):
                dp[l][r] = max(dp[l][r],
                               dp[l][k] + dp[k][r]
                               + b[l] * b[k] * b[r])
    return dp[0][m-1]
```

Complexity: $O(n^3)$ time, $O(n^2)$ space. For large n, additional structure is usually required. Certain special cost functions admit Knuth or divide-and-conquer optimization, but *not every interval DP qualifies*.

Burst Balloons: Why Does the Last Burst Separate the Two Halves?

Consider $[3,1,5,8]$, padded with artificial boundary balloons of value 1. Let $dp[l][r]$ be *the maximum score from removing all balloons between surviving endpoints l and r*. If the final internal balloon burst is k, both l and r still exist, so the last reward is exactly $a_l a_k a_r$. Earlier, the left interval (l,k) and right interval (k,r) can each be optimized independently. The recurrence

$$dp[l][r] = \max_{l<k<r}\{dp[l][k] + dp[k][r] + a_l a_k a_r\}$$

then has a precise physical meaning. If you instead choose k as the first burst, elements across the gap can later become adjacent, preventing a simple independent

sum. This is why **reasoning backward from the last action** is so powerful in interval DP.

6.5 Canonical Problem 3: Longest Palindromic Subsequence (LeetCode 516)

Full problem statement: A subsequence may skip characters

Given a string `s`, find the length of its *longest palindromic subsequence*. You may delete arbitrary characters as long as the remaining characters keep their original relative order. A palindrome reads the same forward and backward. For `s="bbbab"`, the subsequence `bbbb` has length 4; for `"cbbd"`, `bb` has length 2. This is *not* the longest palindromic substring problem, where characters must be consecutive.

Derive the transition from both ends

Let $dp[l][r]$ be the longest palindromic subsequence inside the *closed* interval $s[l : r{+}1]$. If $s[l] = s[r]$, use the equal endpoints around an optimal interior palindrome to obtain $2 + dp[l+1][r-1]$. If the endpoints differ, a palindrome cannot use both as its two outermost characters; discard at least one end and take $\max(dp[l+1][r], dp[l][r-1])$. For `aba`, single-character intervals have value 1; the center `b` has value 1, and the outer a's match, giving $dp[0][2] = 2 + 1 = 3$.

A length-one interval has value 1, and an empty interior has value 0. Iterate by increasing interval length, so every shorter dependency is already available. Check that `aa` produces 2 and `ab` produces 1. Space optimization must preserve the dependencies by coordinating descending l and ascending r; the full two-dimensional table is easier to reason about first.

Problem: What is the length of the longest palindromic *subsequence* of a string? Characters may be skipped. For `bbbab`, the answer is 4, from `bbbb`.

State: $dp[l][r]$ is the longest palindromic subsequence length inside the closed interval $s[l : r+1]$. If the endpoints match, place them around the optimal internal palindrome; otherwise at least one endpoint can be discarded:

$$dp[l][r] = \begin{cases} 2 + dp[l+1][r-1], & s_l = s_r, \\ \max\{dp[l+1][r], dp[l][r-1]\}, & s_l \neq s_r. \end{cases}$$

Base: A single character has length 1; an empty interval has length 0. Process l downward and r upward:

```
def longest_pal_subseq(s):
    n = len(s)
    if not n:
        return 0
    dp = [[0] * n for _ in range(n)]
    for l in range(n - 1, -1, -1):
        dp[l][l] = 1
        for r in range(l + 1, n):
            if s[l] == s[r]:
                dp[l][r] = 2 + (dp[l+1][r-1] if r-l > 1 else 0)
```

```
            else:
                dp[l][r] = max(dp[l+1][r], dp[l][r-1])
    return dp[0][n-1]
```

Complexity: $O(n^2)$ time and space. **Further connection:** The longest palindromic subsequence length equals $LCS(s, reverse(s))$. Thus the same problem can be modeled via either "interval endpoints" or "two-sequence alignment."

6.6 Canonical Problem 4: Matrix Chain Multiplication

Full problem statement: Parenthesization changes computational cost

You must multiply matrices $A_1, \dots, A_n$ in their *fixed order*. Their dimensions are given by $p_0, \dots, p_n$, with A_i of size $p_{i-1} \times p_i$. Matrix multiplication is associative, so parenthesization may change, but the matrix order may *not*. Multiplying an $a \times b$ matrix by a $b \times c$ matrix costs abc scalar multiplications. Minimize the total number of scalar operations. For dimensions 10×30, 30×5, and 5×60, evaluating $(A_1A_2)A_3$ costs $10 \cdot 30 \cdot 5 + 10 \cdot 5 \cdot 60 = 4500$, whereas $A_1(A_2A_3)$ costs $30 \cdot 5 \cdot 60 + 10 \cdot 30 \cdot 60 = 27000$. The optimum is 4500.

Enumerate the position of the final multiplication

Suppose the last operation in the chain $A_l \cdots A_r$ multiplies the result of $A_l \cdots A_k$ by that of $A_{k+1} \cdots A_r$. Each side must already have been computed optimally; otherwise its parenthesization could be improved independently. Define $dp[l][r]$ as the minimal cost for the closed interval $A_l, \dots, A_r$. A single matrix needs no multiplication, so $dp[i][i] = 0$. The two subproducts have dimensions $p_{l-1} \times p_k$ and $p_k \times p_r$, and the last multiplication costs $p_{l-1}p_kp_r$. Minimize over all $l \le k < r$. This yields a standard $O(n^3)$ interval DP: iterate length, then left endpoint, then split position. With zero-based Python dimensions `[10,30,5,60]`, the final-operation cost `dims[l]*dims[k+1]*dims[r+1]` represents exactly the same formula.

Problem: Matrices $A_1, \dots, A_n$ are dimensionally compatible, with A_i of shape $p_{i-1} \times p_i$. Different parenthesizations require different counts of scalar multiplications; minimize the cost. For dimensions $[10, 30, 5, 60]$, calculating A_1A_2 first costs $10 \cdot 30 \cdot 5 + 10 \cdot 5 \cdot 60 = 4500$ multiplications, while calculating A_2A_3 first costs 27000. **State**: $dp[l][r]$ is the minimum cost of multiplying $A_l \cdots A_r$; $dp[i][i] = 0$. Split at k into two subchains:

$$\boxed{dp[l][r] = \min_{l \le k < r} \{dp[l][k] + dp[k+1][r] + p_{l-1}p_kp_r\}.}$$

```
def matrix_chain(dims):
    n = len(dims) - 1
    if n <= 1:
        return 0
    dp = [[0] * n for _ in range(n)]
    for length in range(2, n + 1):
        for l in range(n - length + 1):
            r = l + length - 1
            dp[l][r] = min(
```

```
                dp[l][k] + dp[k+1][r]
                + dims[l] * dims[k+1] * dims[r+1]
                for k in range(l, r)
            )
    return dp[0][n-1]
```

Complexity: $O(n^3)$ time, $O(n^2)$ space. **Fixed interval-DP template:** Iterate interval length first, then left endpoint, then the last decision k, ensuring both smaller subintervals have already been solved.

7 Family Six: State Compression with Bitmask DP

7.1 Recognition: The Set Is Small, but Exactly Which Elements Were Used Matters

Clues: n is usually no more than about 15–20; each task/city may be used or visited only once; order influences costs. A general set-DP state can encode the selected subset as an n-bit integer.

$$mask = \sum_{i=0}^{n-1} b_i 2^i, \qquad b_i \in \{0, 1\}.$$

Use `mask & (1 << i)` to test membership of i, `mask | (1 << i)` to add i, `mask XOR (1 << i)` to remove i when that bit is known to be set, and `mask.bit_count()` for set size. A state may need more than `mask`; sometimes it also needs the current endpoint `last` or a remaining budget.

7.2 Recognition in Detail: Why Must the Set Itself Be a State?

The strongest clue is not a problem title but the combination of **"each object used at most once" and "the order affects future costs."** If the future depends only on how many objects have been processed, ordinary $dp[i]$ suffices. If it depends on *which specific objects* were processed, retain a set. Such problems usually have small n because there are 2^n possible sets.

1. **Visit each city exactly once:** Starting at city 0, the legal next destinations after visiting 1, 3, and 5 differ from those after visiting 1, 2, and 4, even though four cities have been visited in both cases. Store the *visited set*; travel cost also depends on *the current city*, so use $dp[mask][last]$.

2. **Assign people one-to-one to jobs:** If employee 0 takes job 2, nobody else may take job 2. Future possibilities depend on *the set of occupied jobs*. With employees processed in fixed order 0, 1, 2, ..., the count already assigned is `mask.bit_count()`, so another counter is unnecessary.

3. **Choose the smallest team covering required skills:** One person may possess several skills and selecting that person adds them all to the covered set. The state can encode *the set of skills covered*, not the set of people already chosen. The objective is to cover all skills with as few people as possible.

4. **Pair removals and permutations of a small set:** "At each step choose two unused elements to form a pair, with reward depending on which pair number this is." A set mask identifies which people remain; the number of completed pairs is half the number of selected bits.

7.2.1 What Is a Bitmask? A Four-Element Example

Suppose cities are numbered $0, 1, 2, 3$. The visited set containing cities 0 and 2 is binary `0101`, or decimal 5. To test whether city 2 has been visited:

$$5 \;\&\; (1 \ll 2) = 5 \;\&\; 4 = 4 \neq 0.$$

To add city 1:

$$5 \mid (1 \ll 1) = 5 \mid 2 = 7,$$

The resulting binary mask is `0111`. Binary encoding is merely a compact representation of a set; it does not mean the underlying prices or probabilities must themselves be binary.

```
mask = 0b0101             # selected 0 and 2
assert (mask & (1 << 2)) != 0
assert (mask & (1 << 1)) == 0
mask = mask | (1 << 1)  # select 1
assert mask == 0b0111
assert mask.bit_count() == 3
```

7.2.2 When Is mask Insufficient, and Why Add last?

Compare routes $0 \to 1 \to 2$ and $0 \to 2 \to 1$. Both visit set $\{0, 1, 2\}$, so their masks match. But if the next city is 3, the first route departs from city 2 and the second from city 1. Costs `cost[2][3]` and `cost[1][3]` may differ. Thus TSP needs `last`. In assignment, the next employee only needs to know occupied jobs, not the previous employee's job; mask alone suffices.

7.2.3 Estimate State Counts Rather Than Memorizing Formulas

$2^{15} = 32768$ and $2^{20} = 1048576$. TSP additionally tracks the current city n, yielding roughly $n2^n$ states, each enumerating a next city for $O(n^2 2^n)$ time. In Python, a million-row table with 20 objects per row can be memory-intensive. **Seeing $n \leq 20$ makes bitmask DP worth considering, but you must still estimate both states and transitions.**

7.3 Canonical Problem 1: Travelling Salesperson Problem (TSP)

Full problem statement: Visit every city and return to the start

Given n cities and a complete directed travel-cost matrix `cost[u][v]`, start at city 0, visit *every other city exactly once*, and finally return to city 0. Minimize the total tour cost. Costs may be asymmetric: going from u to v need not cost the same as reversing direction. In a four-city symmetric example, the route $0 \to 1 \to 3 \to 2 \to 0$ costs $10 + 25 + 30 + 15 = 80$ and is optimal. If returning to city 0 were *not* required, the problem would instead ask for a Hamiltonian path and require a different terminal condition.

Why both mask and last are indispensable

Suppose visited cities are $\{0, 1, 3\}$. Even with exactly the same visited set, being currently at city 1 makes the cost to next visit city 2 equal to `cost[1][2]`, whereas being at city 3 gives `cost[3][2]`. Therefore the mask alone is insufficient; store the last city as well. For example, binary `1011` means cities 0, 1, and 3 have been visited while 2 has not. After going to city 2, the mask becomes `1111`.

A Top-Down `dfs(mask,last)` can mean the least *remaining additional cost* to visit all remaining cities and return to 0; when the mask is full, its base case must still pay the final return edge. Bottom-Up $dp[mask][last]$ means the least cost *already paid* to reach a given set and endpoint. They compute the same optimum using opposite-looking state definitions. The upper bound is 2^n subsets times n possible endpoints times n next-city choices, or $O(n^2 2^n)$ time. For $n = 20$, a dense Python table of $2^{20} \times 20$ object references may already consume substantial memory; state sparsity and implementation choice matter.

Problem: Given n cities and cost matrix $cost[u][v]$, start at city 0, **visit every city exactly once and return to city 0**, minimizing tour cost. For example, consider the cost matrix

$$\begin{pmatrix} 0 & 10 & 15 & 20 \\ 10 & 0 & 35 & 25 \\ 15 & 35 & 0 & 30 \\ 20 & 25 & 30 & 0 \end{pmatrix}$$

The optimal tour costs 80, for example $0 \to 1 \to 3 \to 2 \to 0$.

State: $dp[mask][u]$ is the minimum cost of a path that starts at 0, visits exactly the cities in $mask$, and ends at u *without yet returning to the origin*. **Base**: $dp[1][0] = 0$, all others $+\infty$. Moving to an unvisited city v gives

$$dp[mask \cup \{v\}][v] = \min\{dp[mask \cup \{v\}][v],\ dp[mask][u] + cost[u][v]\}.$$

At the end, include the edge back to the starting city:

$$\boxed{\min_u \{dp[2^n - 1][u] + cost[u][0]\}.}$$

```python
def tsp_cycle(cost):
    n = len(cost)
    if n <= 1:
        return 0
    INF = float('inf')
    dp = [[INF] * n for _ in range(1 << n)]
    dp[1][0] = 0
    for mask in range(1, 1 << n):
        if not (mask & 1):
            continue
        for u in range(n):
            if dp[mask][u] == INF:
                continue
            for v in range(1, n):
                if mask & (1 << v):
                    continue
                nxt = mask | (1 << v)
                dp[nxt][v] = min(dp[nxt][v],
```

```
                                dp[mask][u] + cost[u][v])
    full = (1 << n) - 1
    return min(dp[full][u] + cost[u][0] for u in range(n))
```

Complexity: $O(n^2 2^n)$ time, $O(n2^n)$ space. **Pitfall:** A shortest Hamiltonian *path* ends with $\min_u dp[full][u]$; a **TSP tour/cycle** must return to the origin, so do not omit that final edge.

The **Top-Down** version uses `dfs(mask,last)` to return the minimum additional cost of visiting all remaining cities and returning to 0:

```
from functools import cache

def tsp_topdown(cost):
    n = len(cost)
    full = (1 << n) - 1
    @cache
    def dfs(mask, u):
        if mask == full:
            return cost[u][0]
        return min(cost[u][v] + dfs(mask | (1 << v), v)
                   for v in range(n) if not mask & (1 << v))
    return dfs(1, 0)
```

Implementation limits: A Python list table of size $n2^n$ uses substantial memory; at $n = 20$ it has more than ten million slots. Depending on the time and memory budget, consider C++, sparse layers, or heuristics.

7.4 Canonical Problem 2: Minimum-Cost Assignment

Full problem statement: Every worker and task must be used exactly once

There are n workers and exactly n tasks. A cost matrix $c[i][j]$ gives the cost of assigning worker i to task j. Assign *exactly one task to every worker* and *exactly one worker to every task*, minimizing the total cost. For $[[9,2,7],[6,4,3],[5,8,1]]$, a minimum assignment sends worker 0 to task 1 (cost 2), worker 1 to task 0 (cost 6), and worker 2 to task 2 (cost 1), for a total cost of 9.

Why assignment DP needs no last dimension

Process workers in fixed order: worker 0 first, then worker 1, and so on. If a bitmask encodes the already-assigned tasks, the next worker is known automatically as $k = \text{popcount}(mask)$. Select an unused task j, add $c[k][j]$ to cost, and mark bit j. Future costs depend on the *current worker and available tasks*, not on which task was assigned to the previous worker, so $dp[mask]$ is sufficient. By contrast, TSP needs `last` because the cost of the next trip depends on the current city.

Hand trace: start at `000`, cost zero. Worker 0 picks task 1: `010`, cost 2. Worker 1 picks task 0: `011`, cost 8. Worker 2 must pick task 2: `111`, total 9. The algorithm also explores the alternatives, retaining the minimum for each assigned-task subset. For the general large-scale Assignment Problem without complicated extra combina-

torial constraints, the Hungarian algorithm is another option; Bitmask DP is particularly transparent for small n.

Problem: There are n employees and n tasks, with assignment cost $c[i][j]$. Assign exactly one task per employee and exactly one employee per task, minimizing total cost. Example:

$$\begin{pmatrix} 9 & 2 & 7 \\ 6 & 4 & 3 \\ 5 & 8 & 1 \end{pmatrix}$$

Optimal: employee $0 \to$ task 1 (cost 2), employee $1 \to$ task 0 (cost 6), and employee $2 \to$ task 2 (cost 1), for total cost 9.

State: $dp[mask]$ is the minimum cost when the tasks in $mask$ have been assigned to the *first* $k = \text{popcount}(mask)$ *employees*. The number already assigned follows from mask size, so no extra i dimension is required.

$$dp[mask \cup \{j\}] = \min\{dp[mask \cup \{j\}], dp[mask] + c[k][j]\}, \ \ j \notin mask.$$

```python
def assignment_min_cost(cost):
    n = len(cost)
    full = (1 << n) - 1
    INF = float('inf')
    dp = [INF] * (1 << n)
    dp[0] = 0
    for mask in range(1 << n):
        if dp[mask] == INF:
            continue
        worker = mask.bit_count()
        if worker == n:
            continue
        for job in range(n):
            if not (mask & (1 << job)):
                nxt = mask | (1 << job)
                dp[nxt] = min(dp[nxt], dp[mask] + cost[worker][job])
    return dp[full]
```

Complexity: $O(n2^n)$ time and $O(2^n)$ space. **Advanced note:** General minimum-cost bipartite matching/assignment admits polynomial-time algorithms such as the Hungarian algorithm, often $O(n^3)$. Thus bitmask DP is not the ultimate algorithm for large, standard assignment instances; it is particularly useful for small n and variants with combinatorial constraints.

Assignment: When Does mask Alone Suffice Without last?

Suppose three employees A, B, C must take three different jobs 0, 1, 2. If employee A takes job 2, the occupied mask is `100`; when assigning B, only the occupied jobs matter. Cost `cost[B][j]` depends on B and the chosen job, not on A's earlier choice beyond its occupancy. Because employee order is fixed, $k = mask.bit_count()$ determines the next employee, and $dp[mask]$ is enough. TSP differs: `cost[last][next]` explicitly depends on the final city, so its state must be $dp[mask][last]$. **The test for removing a state dimension is whether future transitions truly need it.**

7.5 Canonical Problem 3: Smallest Sufficient Team

Full problem statement: Compress skills, not necessarily people

A company requires m distinct skills to complete a project and has several candidate employees. Each candidate possesses a subset of those required skills; multiple candidates may share a skill. Find the *smallest number of people* whose combined skill sets cover every required skill, usually returning the selected indices. For skills `[python,sql,cloud]` and candidates `[[python],[sql],[python,sql],[cloud]]`, selecting candidates 2 and 3 covers all requirements with just two people.

Why the bitmask should encode covered skills

With three skills, let the low, middle, and high bits represent Python, SQL, and Cloud. Someone who knows Python and SQL has mask `011`; someone who knows Cloud has mask `100`. The team's coverage is their bitwise union: `011 | 100 = 111`. Define $dp[mask]$ to store the smallest team covering the skill subset $mask$, along with enough information to recover its members. For each candidate, either skip them or OR their personal skill mask into the current coverage and increase the team size by one. *Each candidate may be used only once*, so the safest approach is to snapshot the DP before processing a candidate and expand only from that snapshot.

If the interviewer asks only for the smallest size, store lengths and predecessor choices rather than copying full team lists for every DP state. Unlike TSP, future success depends on *which skills have been covered*, not on the most recently selected employee, so no `last` dimension is required. When the number of *skills* m is small, 2^m states may be manageable even if there are many candidate people.

Problem: Given m required skills and a set of candidates, each possessing some subset of skills, select the fewest people covering every required skill. Encode each person's skills as $personMask$, and let $dp[mask]$ store the fewest people and corresponding team covering the skills in $mask$.

$$dp[mask \mid personMask] = \min\{dp[mask \mid personMask], 1 + dp[mask]\}.$$

Each person can be selected at most once. Use a snapshot or process states carefully in reverse. Since a bitwise OR may leave the mask unchanged (the person adds no new skill), the safest method is to copy the currently known states before processing each person:

```
def smallest_team(skills, people):
    index = {name: i for i, name in enumerate(skills)}
    masks = []
    for person in people:
        mask = 0
        for name in person:
            if name in index:
                mask |= 1 << index[name]
        masks.append(mask)
    dp = {0: []}
    for i, pm in enumerate(masks):
        for mask, team in list(dp.items()):
            nxt = mask | pm
```

```
            candidate = team + [i]
            if nxt not in dp or len(candidate) < len(dp[nxt]):
                dp[nxt] = candidate
    return dp.get((1 << len(skills)) - 1, [])
```

Complexity: At most 2^m skill sets are possible, and each person may update that many states. Time is roughly $O(P2^m)$, with additional cost for copying teams; stored teams can require up to $O(P2^m)$ space. If only the team size is required, retaining numerical costs and predecessors saves space. **Broader insight:** The compressed dimension can be the number of skills, not necessarily the number of people.

7.6 Algorithmic Limits of This Family

Bitmask DP does not mean "if $n \leq 20$, always use masks." First verify that the *set already selected affects future feasibility or cost*. If an index i already contains sufficient information, avoid exponential states. Conversely, 2^{30} distinct states are impractical even with $O(1)$ work per state. In interviews, explain when combinatorial optimization, graph algorithms, branch-and-bound, or approximations are preferable.

8 Family Seven: Digit DP, Finite Automata, and Markov Counting

8.1 Why Do These Problems Belong Together?

Shared structure: Process a sequence of length n, reading one symbol per step (0/1, heads/tails, or digits on a telephone keypad). The state needs only the *pattern information recognized so far*. Finite-state transitions support three different calculations:

- **Counting:** Use integer addition to accumulate the number of valid sequences.
- **Probability:** Weight transitions by their probabilities to obtain state distributions.
- **Optimization:** If actions can be chosen, use max/min over legal transitions.

To count valid integers in $[0, N]$, additionally track Digit DP's **tight flag (whether the constructed prefix matches the upper bound)** and possibly a leading-zero state. "Markov" here means that a finite state contains all necessary history; it does not imply that every counting problem is stochastic.

8.2 Recognition in Detail: Which Small Piece of History Does a Long Sequence Need?

The stories differ greatly: consecutive coin-toss patterns, legal telephone number sequences, integers below a bound, and finite Markov chains. But all share one property: **we append one symbol at a time, and future legality depends on only a small pattern state**. This is the intuition behind a finite-state automaton.

1. **No three consecutive heads:** "Toss a coin n times without ever seeing HHH." You need not remember every toss, only whether the *trailing run of H* has length 0, 1, or 2. Another H after a run of 2 is forbidden.
2. **Binary integers without adjacent ones:** "Count binary representations in $[0, N]$ without 11." After processing a prefix, remember only whether the last bit was 1; but also remember whether the prefix still equals the upper bound, the *tight* flag. It is distinct from the pattern-memory state.
3. **Keypad dialing paths:** "A chess knight jumps between digit keys; count legal sequences of length n." The next digit depends only on *the current key*, so $dp[t][digit]$ is sufficient.
4. **Extremely long repeated transitions:** "The automaton has just three or five states, but must run for 10^9 steps." If each step applies the same fixed transition matrix, direct simulation is too slow; *matrix exponentiation by squaring* makes the cost logarithmic in the number of steps.
5. **Count integers under a numeric bound:** "Count integers no greater than N

in which a decimal digit never appears or appears at most a fixed number of times." Construct one digit at a time, tracking the position, whether the prefix remains tight, and pattern information; add `started` if leading zeros have special meaning.

8.2.1 Compressing Complete Histories into a Finite State

Suppose the only forbidden pattern is HHH. Prefixes `THTTH` and `HH` have very different histories, but the former ends with a run of one H and the latter with two. All that matters for their future restrictions is the trailing run length. Keeping complete prefixes produces 2^t histories; retaining $k \in \{0, 1, 2\}$ leaves only three states per step.

A state is not a probability. The state is "number of trailing consecutive Hs"; a probability is a weight on a transition edge. For counting, each valid sequence contributes one. For probabilities under a fair coin, each H/T transition gets weight $1/2$. For a biased coin, the weights are p and $1-p$. **Build the same state graph first, then change edge weights and aggregation operations** to solve many variations.

8.2.2 The Meaning of tight in Digit DP

Take decimal upper bound $N = 352$. Construct a number no greater than N, digit by digit from left to right. If the hundreds digit is 2, the prefix is already below 3, so the tens and units digits can freely range from 0 to 9: `tight=False`. If the hundreds digit is 3, the tens digit is limited to 5; if it is also 5, the units digit is limited to 2. The state remembering *whether all previous digits equal the bound* is `tight`. It is not the same as "whether the preceding bit was 1" or "the current run of Hs," so these variables must be tracked separately.

8.2.3 Leading Zeros and the Boundary for Zero or Negative Numbers

Digit DP uses fixed-length representations for values from 0 to N, often padding numbers with zeros. For instance, 7 may be written as `007`. If the task asks for numbers containing exactly two zeros, the two padding zeros usually must not count. Use `started` to mark whether a first nonzero digit has appeared, and decide separately how to handle the number 0 at the end. **The need for started depends on the meaning of leading zeros, not on a rule that all Digit DP must have four dimensions.**

8.3 Canonical Problem 1: Never Toss Three Consecutive Heads

Full problem statement: Replace an entire history with a short memory

Independently toss a fair coin n times, writing H for heads and T for tails. Count sequences of length n that *never contain three consecutive heads* (the substring `HHH`), and compute the probability of such a sequence among the 2^n equally likely outcomes. Every position must be either H or T. For $n = 3$, the only invalid sequence is `HHH`, leaving $2^3 - 1 = 7$ legal sequences. For $n = 5$, the legal count is 24 and the probability is $24/32 = 0.75$.

Why the trailing run length is a sufficient state

Any two histories with the same *number of trailing consecutive H's* permit the same next choices. For instance, `THTH` and `TTTH` both end with exactly one H, so their future constraints coincide. We do not need the entire history. State 0 means no trailing H, state 1 means one trailing H, and state 2 means two. Appending T resets to 0; appending H advances 0 to 1 or 1 to 2; state 2 followed by H is forbidden and creates no transition.

Full time-by-state table before rolling the three states:

```python
def count_without_hhh_table(n):
    dp = [[0] * 3 for _ in range(n + 1)]
    dp[0][0] = 1
    for t in range(1, n + 1):
        dp[t][0] = sum(dp[t - 1])  # append T
        dp[t][1] = dp[t - 1][0]   # append H to 0
        dp[t][2] = dp[t - 1][1]   # append H to 1
    return sum(dp[n])
```

This requires $O(n)$ time and space. Because each time step reads only the previous three entries, it can then be compressed to $(a, b, c) \leftarrow (a + b + c, a, b)$, using $O(1)$ space. If $P(H) = p \neq 1/2$, instead of counting equally likely sequences and dividing by 2^n, assign weights p to H-transitions and $1 - p$ to T-transitions; this directly evolves the probability distribution over the three states.

Problem: Independently toss a fair coin n times. Count the H/T sequences that *never contain HHH*, and find their probability. For $n = 5$, there are 24 valid sequences and probability $24/32 = 0.75$.

State: $dp[t][k]$ counts valid length-t toss sequences *without HHH* whose trailing run of H has length exactly $k \in \{0, 1, 2\}$. **Base**: $dp[0][0] = 1$ and all other entries 0. **Transition:** A tail T resets the trailing-H count:

$$dp[t][0] = dp[t-1][0] + dp[t-1][1] + dp[t-1][2].$$

A head H moves $0 \to 1$ and $1 \to 2$, while $2 \to 3$ is forbidden:

$$dp[t][1] = dp[t-1][0], \qquad dp[t][2] = dp[t-1][1].$$

The answer is the sum of all three states, not just one.

```python
def count_without_hhh(n):
    a, b, c = 1, 0, 0
    for _ in range(n):
        a, b, c = a + b + c, a, b
    return a + b + c

def probability_without_hhh(n):
    return count_without_hhh(n) / (2 ** n)

assert count_without_hhh(5) == 24
```

Complexity: $O(n)$ time and $O(1)$ space. **Generalization:** To prohibit runs of k consecutive Hs, track trailing-H lengths $0, 1, \ldots, k-1$ in a state vector of length k.

8.4 Canonical Problem 2: Markov Transitions and Matrix Exponentiation

Full problem statement: Too many steps for ordinary iteration

Return to counting H/T sequences without `HHH`, but now suppose the length n can be as large as 10^9, and return the count modulo $10^9 + 7$. Even with only three states, performing a billion sequential updates is not suitable for live coding. The key observation is that *the same linear map* takes the fixed-size state vector from step t to step $t + 1$; represent that map by a 3×3 matrix and exponentiate it by repeated squaring.

How one matrix multiplication represents one coin toss

Let $s_t = (a_t, b_t, c_t)^T$, recording numbers of legal sequences with trailing H-run lengths 0, 1, and 2. Appending T to any old state yields state 0, so $a_{t+1} = a_t + b_t + c_t$. Appending H sends old 0 to new 1 and old 1 to new 2, so $b_{t+1} = a_t$ and $c_{t+1} = b_t$. These equalities form a matrix update $s_{t+1} = Ms_t$. Starting from $s_0 = (1, 0, 0)^T$, we have $s_n = M^n s_0$. For $n = 1$, $(1, 1, 0)$ counts sequences T and H; for $n = 2$, $(2, 1, 1)$ sums to four; for $n = 3$, $(4, 2, 1)$ sums to seven.

Matrix exponentiation is *not* a universal acceleration for arbitrary DP. It requires a fixed state dimension and the *same linear transition* at each step. Binary exponentiation takes $O(\log n)$ matrix squarings, giving conventional dense multiplication complexity $O(k^3 \log n)$ for dimension k. A Bellman recurrence involving `max` over adaptive decisions cannot generally be treated as an ordinary matrix product; special algebraic structures, such as max-plus, would need separate analysis.

The preceding rolling transition can be written as a matrix:

$$\begin{pmatrix} a_{t+1} \\ b_{t+1} \\ c_{t+1} \end{pmatrix} = \underbrace{\begin{pmatrix} 1 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}}_{M} \begin{pmatrix} a_t \\ b_t \\ c_t \end{pmatrix}.$$

Thus $s_n = M^n s_0$. If n is very large (say 10^9) and the state dimension k is small, exponentiation by squaring evaluates this in $O(k^3 \log n)$ time, preferable to $O(nk^2)$ step-by-step multiplication.

```
def matmul(A, B, mod):
    n = len(A)
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for k in range(n):
            if A[i][k]:
                for j in range(n):
                    C[i][j] = (C[i][j] + A[i][k] * B[k][j]) % mod
    return C

def matpow(A, p, mod):
    n = len(A)
    R = [[int(i == j) for j in range(n)] for i in range(n)]
    while p:
        if p & 1:
```

```
            R = matmul(R, A, mod)
        A = matmul(A, A, mod)
        p //= 2
    return R

def count_without_hhh_fast(n, mod=1_000_000_007):
    M = [[1, 1, 1], [1, 0, 0], [0, 1, 0]]
    P = matpow(M, n, mod)
    return sum(P[i][0] for i in range(3)) % mod
```

The initial vector is $s_0 = (1,0,0)^T$, so the answer is the sum of the entries in the first column of M^n.

Probability instead of count: Multiply the matrix's transition entries by actual probabilities. For independent fair coin tosses, simply divide the count by 2^n; with $P(H) = p$, give each H edge weight p and each T edge weight $1-p$, treating entry into HHH as absorbing/forbidden probability mass.

From Sequence Counts to Matrix Powers: Accelerating Very Long Horizons

The trailing-H run has only three possible lengths: 0, 1, and 2. If the current valid-sequence counts are (a,b,c), another toss ending in T produces a new length-0 count $a+b+c$; ending with one H requires a previous length-0 state and gives a; ending with two Hs requires a previous length-1 state and gives b. Thus the rolling update is $(a+b+c, a, b)$, with identical coefficients each step.

This is repeated application of matrix M to the state vector: $s_n = M^n s_0$. A naive loop takes n iterations. At $n = 10^9$, square $M^2, M^4, M^8, \ldots$, and use only about 30 exponent bits. Matrix multiplication itself has a cost, so when the state dimension k is large, exponentiation is not necessarily faster than ordinary DP.

8.5 Canonical Problem 3: Digit DP — Binary Integers without Adjacent Ones in $0 \le x \le N$

Full problem statement: A numeric upper bound and a local pattern restriction

For a nonnegative integer N, count integers $x \in [0, N]$ whose *binary representation has no adjacent ones* (no substring `11`). Leading zero padding is harmless for *this* particular property: adding zeros on the left never creates a new `11`. For $N = 13$ (binary `1101`), the valid numbers are $0, 1, 2, 4, 5, 8, 9, 10$, a total of eight. Counting only by bit length is insufficient, because candidates sharing N's bit length may still exceed the upper bound.

Explain tight by hand with $N = 5$, binary 101

To count $0 \le x \le 5$, consider all candidates as three-bit strings. Valid ones are `000`, `001`, `010`, `100`, and `101`: five in total. `011` is within the upper bound but contains adjacent ones, while `110` and `111` are above the bound. Traverse bits from most significant to least significant. If the chosen prefix exactly matches the prefix of `101`, we are `tight=True`, and the current bit must not exceed the corresponding upper-bound bit.

Once the chosen prefix is strictly smaller, `tight=False`, and later bits can freely be 0 or 1. At the same time, `prev1` records whether the previous bit was one, forbidding a subsequent adjacent one.

After all bit positions have been processed without violating either condition, return 1, since the chosen bit sequence represents one valid integer. To count the interval $[L, R]$, use $F(R) - F(L-1)$ and define $F(-1) = 0$. For constraints involving the *actual beginning* of the numeric representation, such as forbidding two adjacent zeros, left padding could create spurious `00`; introduce a separate `started` flag to distinguish leading padding from actual digits.

Problem: Count nonnegative integers in $[0, N]$ whose binary representations never contain substring 11. For $N = 13$, valid values are $0, 1, 2, 4, 5, 8, 9, 10$, giving answer 8.

Why ordinary one-dimensional DP is insufficient: When constructing an integer bit by bit, the chosen prefix may already be below N, or it may still equal the corresponding prefix of N. If still equal, the current bit is bounded by the corresponding bit of N. Track this with `tight`:

- *tight* = *True*: Every previous bit matched N; the current bit cannot exceed the corresponding bit of N.
- *tight* = *False*: An earlier bit was smaller; the current bit may freely be 0 or 1.

Also record `prev1`, forbidding two successive ones. Leading zeros cannot create an illegal 11 here, so pad all numbers to the binary length of N *without an additional started state*.

```python
from functools import cache

def count_no_adjacent_ones(N):
    if N < 0:
        return 0
    bits = bin(N)[2:]
    @cache
    def dfs(pos, tight, prev1):
        if pos == len(bits):
            return 1  # one complete number
        upper = int(bits[pos]) if tight else 1
        total = 0
        for bit in range(upper + 1):
            if prev1 and bit:
                continue
            new_tight = tight and (bit == int(bits[pos]))
            total += dfs(pos + 1, new_tight, bit)
        return total
    return dfs(0, True, 0)

assert count_no_adjacent_ones(13) == 8
```

Complexity: $O(\log N)$ layers, with only constant-sized `tight` and `prev1` states per layer; therefore $O(\log N)$ time and space. To count values in $[L, R]$, usually compute $F(R) - F(L-1)$.

8.6 When Is started Required for Leading Zeros?

When a problem depends on *the first true digit, the number of digits, adjacent equal digits, or how often a digit occurs*, left-padding with zeros may change the answer. For example, when counting decimal integers with no equal adjacent digits, multiple padded zeros on the left are not real adjacent zero digits. Include `started` to indicate whether the first nonzero digit has appeared.

Standard template: `dfs(pos,tight,started,automaton_state)`. If not started and the chosen digit is 0, keep `started=False` without applying the ordinary pattern transition. Otherwise update the automaton using the real digit. How to treat the all-zero integer at termination depends on whether the task includes zero.

8.7 Canonical Problem 4: Knight Dialer (LeetCode 935)

Full problem statement: Knight moves as transitions on a small finite graph

Imagine the familiar telephone keypad, with digits 1–9 laid out as `1 2 3 / 4 5 6 / 7 8 9` and digit 0 below. A chess knight starts on *any* digit key and must use a standard knight move (two units along one axis, one along the other) to land on an existing digit key at each step; positions outside the keypad are forbidden. Count digit sequences of length n that can be formed, usually modulo $10^9 + 7$. For length 1 there are 10 possibilities, one per starting digit. Digit 5 has no valid knight move and thus cannot be extended to a longer sequence.

Why ten end-position states replace the entire number sequence

Future legal moves depend only on the *current digit*, not on earlier visited digits: revisiting keys is allowed. Define $dp_t[d]$ as the number of length-t sequences *ending at digit* d. Initially $dp_1[d] = 1$ for all ten digits. For each digit d, iterate over its legal knight neighbors to and add $dp_t[d]$ to $dp_{t+1}[to]$. For example, from 1 the knight can jump to 6 or 8; from 0 it can jump to 4 or 6. Use a *fresh* array for each new length, so updates for step $t + 1$ cannot be reused within that same step to generate invalid longer walks.

The transition graph contains only ten vertices, so simple iteration requires $O(n)$ time and $O(1)$ auxiliary space. Only if n is extremely large is it worthwhile to encode the graph as a 10×10 transition matrix and use fast exponentiation.

Problem: A chess knight jumps between keys on a telephone keypad. Starting at any digit, form a length-n digit sequence, and count all legal sequences (usually modulo $10^9 + 7$).

State: $dp_t[d]$ is the number of sequences of length t ending in digit d. Initially $dp_1[d] = 1$. For each move from a key to a reachable next key, add the source count to the destination. The keypad connections form a *directed transition graph*; the problem is finite-state path counting.

```
def knight_dialer(n, MOD=1_000_000_007):
    moves = [[4, 6], [6, 8], [7, 9], [4, 8],
             [0, 3, 9], [], [0, 1, 7], [2, 6],
```

```
            [1, 3], [2, 4]]
    dp = [1] * 10
    for _ in range(2, n + 1):
        nxt = [0] * 10
        for digit in range(10):
            for to in moves[digit]:
                nxt[to] = (nxt[to] + dp[digit]) % MOD
        dp = nxt
    return sum(dp) % MOD
```

Complexity: A constant number of edges per step gives $O(n)$ time and $O(1)$ space. For huge n, write the keypad transitions as a 10×10 matrix and exponentiate, taking $O(10^3 \log n)$ time.

8.8 Advanced Pattern Matching: What Does an Automaton State Represent?

If the task forbids substring `HTHT`, tracking only the trailing run of H is insufficient; prefixes HT or HTH may already be partially matched. Define the state as **the length of the longest suffix matching a prefix of the target pattern** (the idea behind a KMP prefix automaton). Update that length upon reading each symbol. A full match enters a forbidden or absorbing state. Counting, probability calculation, and optimal stopping can all be interpreted as DP or Markov decision processes over finite states.

9 Four Steps for Unfamiliar Problems: From Words to Recurrences to Optimization

9.1 Step One: Use Input Constraints to Estimate an Acceptable Complexity

Do not immediately guess `dp[i][j]`. First inspect n, capacity bounds, and state-value ranges. Estimate a feasible runtime and work backward to ask whether the expected number of states fits. The following table provides **practical clues, not hard rules**:

Input size clue	Candidate complexity	Common structure
$n \le 15 \sim 20$	$O(n2^n), O(n^2 2^n)$	Visited-set, assignment, TSP bitmasks
$n \le 100 \sim 500$	$O(n^3)$ (time-limit dependent)	Interval splits, three-dimensional DP
$n \le 10^3 \sim 5 \cdot 10^3$	$O(n^2)$ (constant-factor dependent)	Two sequences, naive LIS, two-dimensional states
$n \le 10^5 \sim 10^6$	$O(n)$ or $O(n \log n)$	Constant-state machines, rolling DP, greedy/binary-search optimization
$n \ge 10^9$	$O(\log n)$, etc.	Small-state matrix powers, Digit DP, mathematical derivations

Use these clues carefully: Memory limits, implementation language, branching factors, and actual reachable states matter. Even with $n = 1000$, $O(nW)$ is unrealistic if capacity is one million; similarly, $O(n^2 2^n)$ for $n = 20$ may already be expensive in Python.

9.2 Step Two: Use the Amnesia Test to Find a Minimal Sufficient State

Stand at step i, forget the entire previous path, and ask: What else must I know to determine *legal next actions and optimal future value*?

- At most K trades ⇒ number completed k;
- Whether I can buy depends on a previous sale ⇒ cooldown status;
- No three consecutive long positions ⇒ current action and streak length;
- Visit each city at most once ⇒ set *mask*;

- Align two character sequences ⇒ two prefix positions (i, j);
- Random draws without replacement ⇒ remaining category counts (r, b).

On paper, write **"Which historical facts affect the future?"** Put each into the state. Then attempt to remove redundant dimensions: if accumulated winnings can be reconstructed from remaining red and black counts, no extra profit dimension is needed; if processed employee count is `mask.bit_count()`, do not store another i.

9.3 Step Three: Write the Natural Recursive Actions, Then Cache Repeated States

A generic maximization recursion has the following skeleton:

```python
from functools import cache

@cache
def dfs(state):
    if terminal(state):
        return terminal_value(state)
    best = float('-inf')
    for action in legal_actions(state):
        next_state, reward = transition(state, action)
        best = max(best, reward + dfs(next_state))
    return best
```

This skeleton illustrates the idea; `terminal` and `transition` must be defined for the actual problem. If an action is followed by a random outcome, write instead

$$Q(s,a) = \sum_{s'} P(s' \mid s,a)\,[r(s,a,s') + V(s')], \quad V(s) = \max_a Q(s,a).$$

For feasibility, combine branches with OR; for minimization, use min; for counting, add. `@cache` is valid only with a sufficient state key, manageable state count, and no unhandled recursion cycles. Python recursion at $n \sim 10^5$ can fail due to call-stack depth even when memoization is logically correct.

9.4 Step Four: Convert to Bottom-Up and Reduce Space

Follow this mechanical process:

1. Check whether `dfs(i,...)` depends on $i+1$ or $i-1$, and choose an evaluation order opposite to the recursive dependency.
2. Determine whether each state depends on the *next time layer* or the *previous time layer*; use separate `old/new` states to prevent same-iteration contamination.
3. If each layer depends on only a constant number of neighboring layers, roll the table down to $O(\#\text{states per layer})$ space. Do not automatically claim $O(1)$.
4. If actual actions must be returned, store the selected action or predecessor for each state, or design a way to reconstruct it. Optimal values alone may be insufficient to recover a full path.

Key Insight

Four-step summary: Estimate complexity budget → use the amnesia test to define state → enumerate actions in a recursion → fill/roll the table in dependency order.

9.5 Five Trap-Like Unfamiliar Questions: Recognizing States on the Spot

1. **"You can draw only twice more and red cards pay money":** Clarify whether stopping is allowed after a draw, whether cards are replaced, and whether accumulated winnings can be wiped out. These determine whether the state includes *remaining attempts, remaining ball counts, and current wealth*; then compare stop with expected continuation.
2. **"Choose nonadjacent tasks, at most three tasks total":** Start from House Robber, then add selected count k to obtain $dfs(i,k)$. Taking the current task leads to $dfs(i+2,k+1)$, skipping to $dfs(i+1,k)$.
3. **"Each resource may be used twice; count ways to make exactly 11":** This is bounded-knapsack counting. Do not use the standard unbounded forward capacity loop, which might select the same resource more than twice. Start with a two-dimensional recurrence explicitly enumerating 0, 1, or 2 copies of each resource, then consider space optimization.
4. **"Consecutive segments of a numerical series may be merged; the cost depends on segment endpoints":** A prefix $dp[i]$ is not necessarily sufficient. Try an interval state $dp[l][r]$ and ask whether the last merge split point makes the two sides independent.
5. **"Choose the next destination among cities not yet visited":** Even with the same number visited, the *specific visited set* affects legal next destinations; if travel cost depends on the current city, store it too. Write $dfs(mask, last)$ and check whether $n2^n$ states are feasible.

Practice habit: Before writing code, try to state in 30 seconds why every state dimension is necessary, what the base case is, whether transitions partition by the last step or current action, and whether the answer combines candidates via max, min, Boolean logic, counting, or expectation. Once the verbal description is right, the code is usually much easier.

9.6 From Mathematical Recurrence to Submission-Ready Code: How to Explain It

Explain DP to the interviewer in this order: *"First, I define the state ... Then I split the possible decisions ... These cases cover all valid solutions ... The recurrence follows ... I initialize the base cases ... I calculate states in this order ... The complexity is ..."* This communicates more genuine algorithmic understanding than opening with

"This is two-dimensional DP." A **correctness proof** must show that the decision cases are exhaustive and nonoverlapping and that the remainder retains the defined state semantics. A **complexity argument** counts the possible values of each dimension and multiplies by the work per state.

If the interview requires the *optimal action sequence*, not merely its value—for example, identifying which projects were selected or which days to trade—you usually also need to **record the argmax action** or backtrack through the full DP table. Compressing space to $O(1)$ may discard information needed to reconstruct an entire policy. Explain that trade-off when optimizing.

9.7 An Eight-Step Checklist for Every Unfamiliar Interview Problem

1. Construct a tiny example with two or three elements and **list all legal actions by hand** to avoid misunderstanding the rules.
2. State the objective in one precise sentence: maximum payoff, fewest operations, reachability, number of ways, or risk-neutral expectation?
3. Identify each action available now and the **timing of information**; especially distinguish choosing before and after stochastic observations.
4. Perform the **amnesia test**: remove history irrelevant to the future and preserve every summary needed for legal actions and future rewards.
5. State the exact return meaning of `dfs(state)` and specify base cases and invalid states.
6. Enumerate actions and write the immediate reward, next state, and aggregation operator; hand-check two or three states.
7. Estimate **reachable state count** $\times$ **branches per state**, then choose top-down caching or bottom-up tabulation.
8. If asked to optimize, analyze time/space bottlenecks, rolling-update dependencies, whether the policy is recoverable, and a brute-force comparison test on tiny cases.

9.8 Complete Unfamiliar Quant Case: Restricted High-Frequency Trading Decisions

9.8.1 Restating the Problem and Mathematical Model

Given predicted P&L signals $pnl[0], \ldots, pnl[n-1]$ for the next n seconds (possibly positive or negative), choose one action each second: **long L** (earn $+pnl[t]$), **short S** (earn $-pnl[t]$), or **flat F** (earn 0). Rules: (1) never be long or short in the same direction for three consecutive seconds; (2) switching directly from L to S or S to L in consecutive seconds incurs cost $cost \geq 0$; placing at least one flat second between them avoids that cost. Maximize total net P&L, with n potentially as large as 10^5.

This is a **deterministic, discrete-time control problem with a fully specified future sequence of model-predicted payoffs**. In live markets the future P&L sequence is not known with certainty; this interview model is an algorithm exercise, not a risk-free arbitrage strategy.

9.8.2 Step 1: Use the Input Size to Target an $O(n)$ State Machine

With $n \le 10^5$, we cannot enumerate 3^n action sequences. Since only a fixed set of action constraints applies each second, seek a constant number of status states.

9.8.3 Step 2: The Amnesia Test

Before deciding at time t, we need only:

- Previous action $last \in \{-1, 0, 1\}$, to determine reversal cost;
- Number of consecutive seconds of that action $streak \in \{0, 1, 2\}$, to enforce streak limits;
- Current time t, to read $pnl[t]$.

Thus $dfs(t, last, streak)$ is sufficient, and only five combinations of $(last, streak)$ are legal:

$$(0,0), \quad (1,1),(1,2), \quad (-1,1),(-1,2).$$

Here 0 means flat, 1 means long, and -1 means short.

9.8.4 Step 3: Turn Brute-Force Actions into Memoized Recursion

State: $dfs(t, last, streak)$ is the *maximum additional payoff* from second t onward, conditional on the preceding action and its consecutive duration. **Base**: At $t = n$, return 0. At each step, enumerate flat, long, and short, enforce legality, and deduct switching costs:

```python
from functools import cache

def trading_topdown(pnl, cost):
    n = len(pnl)
    @cache
    def dfs(t, last, streak):
        if t == n:
            return 0
        best = dfs(t + 1, 0, 0)   # flat
        if not (last == 1 and streak == 2):
            next_s = streak + 1 if last == 1 else 1
            penalty = cost if last == -1 else 0
            best = max(best, pnl[t] - penalty
                       + dfs(t + 1, 1, next_s))
        if not (last == -1 and streak == 2):
            next_s = streak + 1 if last == -1 else 1
            penalty = cost if last == 1 else 0
            best = max(best, -pnl[t] - penalty
                       + dfs(t + 1, -1, next_s))
        return best
    return dfs(0, 0, 0)
```

Correctness: Every valid sequence must choose one of these three actions at time t. For each action, immediate payoff and the next state are uniquely determined; replacing the remainder by its optimal continuation preserves legality. Maximizing over the three possibilities therefore yields the optimal value. **Complexity**: $O(5n \cdot 3) = O(n)$ time and cache space, but Python recursion depth is unsuitable for large n, so use iteration.

9.8.5 Step 4: Roll Five Forward States into $O(1)$ Space

Let F, L_1, L_2, S_1, S_2 denote the *best cumulative payoffs* at the end of the current second in the five legal states. Initialize $F = 0$ and all others unreachable ($-\infty$). For current $p = pnl[t]$:

$$
\begin{aligned}
F' &= \max\{F, L_1, L_2, S_1, S_2\}, \\
L_1' &= p + \max\{F, S_1 - cost, S_2 - cost\}, \\
L_2' &= p + L_1, \\
S_1' &= -p + \max\{F, L_1 - cost, L_2 - cost\}, \\
S_2' &= -p + S_1.
\end{aligned}
$$

Why does L_1' not depend on old L_2? That would represent a third consecutive long second, violating the restriction. A new L_1 must enter from flat or short by a *switch*. Meanwhile, L_2' can only come from L_1, representing exactly one consecutive previous long second. The analogous rule holds for shorts.

```python
def trading_iterative(pnl, cost):
    neg = float('-inf')
    F, L1, L2, S1, S2 = 0, neg, neg, neg, neg
    for p in pnl:
        newF = max(F, L1, L2, S1, S2)
        newL1 = p + max(F, S1 - cost, S2 - cost)
        newL2 = p + L1
        newS1 = -p + max(F, L1 - cost, L2 - cost)
        newS2 = -p + S1
        F, L1, L2, S1, S2 = newF, newL1, newL2, newS1, newS2
    return max(F, L1, L2, S1, S2)
```

Complexity: $O(n)$ time and $O(1)$ auxiliary space. **Tiny examples:** With predicted payoffs $[3, 3, 3]$ and switching cost 2, three consecutive long seconds are illegal; choose L,L,F for profit 6 (or L,F,L for 6), and the optimum is 6. With $pnl = [3, -3]$ and switching cost 2, choose L then S for $3 + 3 - 2 = 4$, exceeding L then F for 3.

9.8.6 Interview Validation: Brute-Force Small Samples

For $n \leq 8$, enumerate all 3^n action sequences, checking streak length and reversal costs one by one, then compare with iterative DP. For unfamiliar state machines, randomized tiny-instance *property testing* often reveals missing state information more effectively than checking a single hand-picked example.

```python
from itertools import product

def trading_bruteforce(pnl, cost):
```

```
    best = float('-inf')
    for actions in product((-1, 0, 1), repeat=len(pnl)):
        last, streak, score, ok = 0, 0, 0, True
        for a, p in zip(actions, pnl):
            ns = (streak + 1 if a == last else 1) if a else 0
            if ns > 2:
                ok = False
                break
            score += a * p
            if a and last and a != last:
                score -= cost
            last, streak = a, ns
        if ok:
            best = max(best, score)
    return best
```

9.9 Troubleshooting: Six Ways to Get Stuck in an Interview

1. **Cannot identify the state:** Begin with uncached recursion; find where two paths reach the same future subproblem, and use the amnesia test to add missing variables.
2. **Unclear loop order:** Draw dependency arrows between states and evaluate in DAG topological order. Iterate intervals by length; choose knapsack capacity direction according to usage limits.
3. **Double-counted results:** State explicitly whether order distinguishes solutions. Prove that transition cases are disjoint and exhaustive.
4. **Incorrect initialization:** Is capacity “at most” or “exactly filled”? Are we maximizing or minimizing? First write a mathematical representation for unreachable states.
5. **Cycles appear:** Memoization alone does not solve cyclic dependencies. Consider DAG structure, finite horizons, value iteration, or linear systems.
6. **Space optimization follow-up:** Prove only a bounded number of neighboring layers are needed before rolling. If the optimal policy must be reconstructed, retain predecessor information or devise a reconstruction method.

Appendix A Interview Quick Reference: Seven DP Families

Recognition label	Typical state	Key transition or action
Stop / Continue	$V(t, s)$	$\max\{\text{Stop}, \mathbb{E}[\text{Continue}]\}$
Linear Take/Skip	$dp[i]$	$\max\{dp[i-1], dp[i-2] + a_i\}$
Trading FSM	$dp[t][s][k]$	Each legal old status → new status, adding edge reward
0/1 Knapsack	$dp[c]$	Descending: $\max(dp[c], dp[c - w] + v)$
Unbounded	$dp[c]$	Ascending: min/max or additive counting
Two Sequences	$dp[i][j]$	Match final characters or consume one tail
Grid Paths	$dp[i][j]$	$dp[i-1][j] + dp[i][j-1]$ (valid cells)
Interval Game	$dp[l][r]$	$\max\{a_l - dp[l+1][r], a_r - dp[l][r-1]\}$
Interval Split	$dp[l][r]$	Enumerate final split k, combine left/right subintervals
Bitmask Visit	$dp[mask][last]$	Enumerate unvisited v and set bit v
Digit DP	$dfs(pos, tight, state)$	Enumerate current digit; update tight/pattern state
Markov Counting	$dp[t][state]$	Follow automaton edges; sum counts or weight by probabilities

A.1 Complexity Quick Reference

- Linear finite-state DP: $O(nS)$, or $O(n)$ when S is constant; roll to $O(S)$ space.
- 0/1 and unbounded knapsack: $O(nW)$ time, $O(W)$ space; capacity-based DP is pseudo-polynomial.
- Two sequences: $O(mn)$ time; value-only space is often reducible to $O(\min(m, n))$.

- End-taking games: $O(n^2)$ time; split-point triple loops often take $O(n^3)$.
- Bitmask assignment: $O(n2^n)$; bitmask TSP: $O(n^2 2^n)$.
- Fixed small automaton counting: $O(nE)$; for huge horizons use $O(S^3 \log n)$ matrix exponentiation.

Appendix B Pre-Interview Self-Test: From Recognition Clues to Code

B.1 Ten Quick Questions You Must Be Able to Answer without Coding

1. Why is the continue value in the dice game not simply the next roll's mean of 3.5?
2. How should $V(r, b)$ be defined in red/black sampling to avoid double-counting past winnings?
3. Why does the take branch of House Robber begin at $i + 2$?
4. Why must a stock-trading state record whether you hold a share or are in cash?
5. Why does one-dimensional 0/1 knapsack scan capacities downward? What goes wrong if it scans upward?
6. Why do Coin Change II and Combination Sum IV use different loop orders?
7. How is LCS state $dp[i][j]$ different from LIS state $dp[i]$?
8. Why does $a_l - dp[l + 1][r]$ have a minus sign in an end-taking coin game?
9. If TSP requires returning to the starting city, which final cost must be added?
10. What do tight and started mean in Digit DP?

B.2 Suggested Answers to the Ten Questions

1. After observing the next outcome, one can make further stop/continue decisions; use the next state's *optimal value*, not just one immediate reward's mean.
2. Incremental-value definition: stop pays 0 and transitions add ± 1. Total-value definition: stopping pays accumulated winnings and transitions do not add that payout again.
3. Taking the current item forbids its immediate neighbor, so the remainder begins two positions later.
4. Holding and cash have different legal future actions and cash flows; they cannot be merged into one state.
5. Descending order makes the current item's take branch read the preceding item-layer's state, ensuring one use only.
6. The former counts unordered combinations with items outside; the latter counts ordered sequences with amount outside, enumerating the last element.
7. LCS involves prefixes of two separate sequences; LIS must end at a specific position in one array.

8. The opponent moves next; their maximum net advantage is the negative of my corresponding net advantage, so I subtract it.
9. The edge from the last visited city back to origin 0 after all other cities have been visited.
10. tight tracks whether the current digit is constrained by the upper bound; started distinguishes actual digits from leading padding zeros.

Appendix C General Python Templates and Self-Checks

C.1 Top-Down Optimal-Value Template

```python
from functools import cache

def solve(data):
    @cache
    def dfs(state):
        if terminal(state):
            return base_value(state)
        ans = float('-inf')
        for action in actions(state):
            if valid(state, action):
                reward, nxt = move(state, action)
                ans = max(ans, reward + dfs(nxt))
        return ans
    return dfs(initial_state(data))
```

This is a pseudocode skeleton: `terminal/actions/valid/move` are not Python built-ins and must be defined for each problem. For counting or reachability, change the aggregation operation.

C.2 Bottom-Up State-Machine Template

```python
def state_machine(sequence, states, initial):
    NEG = float('-inf')
    dp = {s: NEG for s in states}
    dp[initial] = 0
    for obs in sequence:
        new_dp = {s: NEG for s in states}
        for s, value in dp.items():
            if value == NEG:
                continue
            for nxt, reward in allowed_transitions(s, obs):
                new_dp[nxt] = max(new_dp[nxt], value + reward)
        dp = new_dp
    return max(dp.values())
```

This is likewise a *conceptual template;* `allowed_transitions` must be defined by the problem. Using old and new tables at each step is usually easiest to prove correct for a complex state machine.

C.3 Pre-Submission Checklist

- Specify whether dp means *at most* or *exactly*, *prefix optimal* or *must end here*, and *future incremental reward* or *total payoff*.
- Base cases: empty input, zero capacity, all-negative data, one item, unreachable states, and a zero target.
- Direction: Does loop order ensure prerequisite subproblems are already computed? Can space compression accidentally reuse an item?
- Stochastic timing: Do transition probabilities sum to one, and when is the decision made relative to random observations?
- Games: Is the objective your own total score, the players' score difference, or a replicated terminal payoff?
- Implementation: Deep recursion, integer overflow (Python big integers can still consume memory), infinity and negative-infinity initialization, and cache size limits.
- Complexity: Do not simply count loops. Multiply the number of states by cost per transition and recognize pseudo-polynomial algorithms.

The Final Problem-Solving Habit

First decide what must be remembered, then what actions are possible; solve the subproblems before organizing the computation order.

ABOUT THE AUTHOR

Mizuki Seika

Quantitative Trader · Web3 Explorer · Independent Idol & Music Producer

I'm Mizuki Seika, a quantitative trader working at the intersection of mathematics, financial markets, and emerging technology; an explorer of Web3 and cryptocurrency; and, in a parallel universe that happens to be equally real, an independent idol and original music producer.

My academic path spans Carnegie Mellon University's Master of Science in Computational Finance (MSCF '28), Mathematics at Tsinghua University ('25), and an exchange year at the University of Oxford ('24). My interests range from quantitative research and systematic trading to decentralized finance and digital assets.

To me, markets and music have more in common than they appear to: both reward an ear for signals, a sense of structure, and the courage to build something original.

> *In finance, I turn complexity into models.*
> *In music, I turn it into melodies.*

This work is where I bring that same creative discipline to learning --- reconstructing knowledge until the pieces form a framework that actually makes sense.

Explore My Worlds

LinkedIn
Professional profile

GitHub
Open-source projects and research

Spotify
Original music

Instagram
The creative side, on and off stage

Thank you for reading.